



République Islamique de Mauritanie

Université de Nouakchott

Faculté des Sciences et Techniques

Département de Mathématiques

DIPLÔME DE LICENCE EN MATHÉMATIQUES

FILIÈRE : MI

Thème :

**Optimisation des tournées de livraison à fenêtres de
temps dans le cadre d'un système de gestion du
transport : conception, réalisation et évaluation**

Réalisé par :

Abderrahmane Abdellahi Beyah C25411

Encadré par :

Dr. Ahmed Sejad

Année Universitaire 2025–2026

À mes chers parents,

*qui m'ont toujours soutenu et encouragé
tout au long de mon parcours académique.*

À ma famille,

pour leur patience et leur confiance indéfectible.

Remerciements

Mes remerciements les plus sincères s'adressent à mon encadrant, **Dr. Ahmed Sejad**, pour la confiance qu'il m'a accordée en acceptant de diriger ce projet. Ses conseils avisés et sa disponibilité ont été déterminants dans l'aboutissement de ce travail. Ses orientations m'ont permis de structurer ma démarche et d'approfondir ma réflexion, tant sur le plan théorique que pratique.

Je tiens à remercier les membres du jury pour l'honneur qu'ils me font en acceptant d'examiner et d'évaluer ce travail. Leurs remarques et suggestions contribueront à enrichir cette étude.

Ma reconnaissance va également à l'ensemble des enseignants du **Département de Mathématiques** de la Faculté des Sciences et Techniques de l'Université de Nouakchott. Les connaissances acquises en mathématiques, en informatique et en recherche opérationnelle au cours de ces trois années de licence constituent le socle sur lequel repose ce projet.

Je remercie chaleureusement mes parents pour les sacrifices qu'ils ont consentis afin de me permettre de poursuivre mes études. Leur soutien inconditionnel, leurs encouragements dans les moments difficiles et leur confiance en moi ont été ma principale source de motivation. Ce travail est avant tout le fruit de leur investissement.

Je remercie ma famille, mes frères et sœurs, pour leur présence constante et leur patience tout au long de ce parcours.

Mes remerciements s'adressent aussi à mes amis et camarades de promotion pour les échanges, l'entraide et les moments partagés durant ces années universitaires.

Enfin, je remercie toute personne ayant contribué, de près ou de loin, à la réalisation de ce travail.

Résumé

La logistique du dernier kilomètre représente une part importante des coûts logistiques [6]. Ce projet de fin d'études présente la conception et la réalisation d'un Système de Gestion du Transport (TMS) intégrant un moteur d'optimisation mathématique pour résoudre le problème de tournées de véhicules avec fenêtres de temps (VRPTW) dans le contexte mauritanien.

Le VRPTW est un problème NP-difficile dont l'espace de solutions croît de manière factorielle ($O(n!)$). Pour 50 clients, il existe plus de 3×10^{64} arrangements possibles, rendant toute approche manuelle impraticable.

Le système implémente un solveur basé sur la métaheuristique *Guided Local Search* via Google OR-Tools (temps de calcul : 30s-4.5min selon la taille). L'application web permet la gestion des commandes avec contrôle d'accès par rôle (expéditeur, dispatcheur, chauffeur, administrateur). L'architecture sépare l'API (backend FastAPI) du calcul d'optimisation (worker Celery asynchrone) avec base PostgreSQL et frontend React.

L'implémentation utilise OSRM auto-hébergé pour le calcul matriciel (requête unique, latence $< 1\text{ms}$, sans coût API). Le solveur prend en compte les contraintes de capacité (poids/volume), fenêtres de temps, types de véhicules, et multi-trajets. Les données cartographiques proviennent d'OpenStreetMap Mauritanie.

Les tests de performance montrent : temps de calcul de 30s pour petites instances (< 20 commandes), 1-3 min pour instances moyennes (20-50 commandes), et 2.5-4.5 min pour grandes instances (50-100+ commandes), augmentation mémoire durant l'optimisation < 20 MB (métrique mesurée : delta RSS du processus Celery, données : 31 exécutions enregistrées), taux de service $> 85\%$ lorsque la flotte et les chauffeurs disponibles couvrent la demande, et API capable de supporter 50-670 requêtes/seconde selon la complexité des endpoints (benchmark avec Apache Bench, 100 à 1000 requêtes par endpoint). Le système est déployé en production sur serveur VPS avec pipeline CI/CD automatisé via GitHub Actions.

Mots-clés : VRPTW, Optimisation combinatoire, Recherche opérationnelle, Guided Local Search, OR-Tools, Système de gestion du transport, Architecture distribuée, FastAPI, React, Docker.

Table des matières

Résumé	3
Liste des figures	7
Liste des tableaux	8
Liste des abréviations	9
Introduction Générale	10
1 Analyse des besoins et Étude préalable	13
1.1 Introduction	13
1.2 Présentation de la problématique	13
1.2.1 La complexité de la livraison du dernier kilomètre	13
1.2.2 Limites des approches manuelles	14
1.2.3 Contexte opérationnel mauritanien	14
1.2.4 Formulation du problème : VRPTW multi-contraint	15
1.3 Présentation de la solution proposée	16
1.3.1 Vision générale	16
1.3.2 Architecture globale	16
1.3.3 Flux fonctionnel principal	17
1.4 Identification des besoins fonctionnels	18
1.4.1 Synthèse des besoins fonctionnels	18
1.5 Identification des besoins non fonctionnels	21
1.5.1 Performances	21
1.5.2 Fiabilité et disponibilité	21
1.5.3 Sécurité	22
1.5.4 Utilisabilité et maintenabilité	22
1.6 Conclusion	23
2 Conception Globale et Modélisation	24
2.1 Introduction	24

2.2	Conception Logicielle (UML)	25
2.2.1	Diagramme de cas d'utilisation	25
2.2.2	Diagramme de classes	27
2.2.3	Diagrammes de séquence	27
2.3	Conception Mathématique : Le Modèle VRPTW	32
2.3.1	Vue d'ensemble du modèle	32
2.3.2	Ensembles et indices	33
2.3.3	Paramètres	33
2.3.4	Contraintes de routage et de degré	36
2.3.5	Utilisation des véhicules et degré au dépôt	36
2.3.6	Contraintes de capacité	37
2.3.7	Propagation temporelle et journée de travail	37
2.3.8	Fenêtres de temps — mode dur (par défaut)	38
2.3.9	Fenêtres de temps — mode souple (optionnel)	38
2.3.10	Contrainte de type de véhicule (régime de température)	39
2.3.11	Domaines des variables	39
2.3.12	Correspondance entre modèle et implémentation	44
2.4	Conclusion	44
3	Réalisation et Évaluation	46
3.1	Introduction	46
3.2	Architecture Technique et Choix Technologiques	46
3.2.1	Vue d'ensemble de l'architecture	46
3.2.2	Stack Backend	47
3.2.3	Stack Frontend	47
3.2.4	Base de Données et Persistance	48
3.2.5	Services Externes	48
3.3	Implémentation de l'Optimisation	49
3.3.1	Architecture du Moteur d'Optimisation	49
3.3.2	Optimisation Réseau : Réduction de Complexité	50
3.3.3	Implémentation du Solveur OR-Tools	50
3.3.4	Gestion des Erreurs et Cas Limites	52
3.4	Interfaces Utilisateur	52
3.4.1	Vue d'ensemble	52
3.4.2	Interface Expéditeur	52
3.4.3	Interface Dispatcheur	53
3.4.4	Interface Chauffeur	56
3.4.5	Interface Administrateur	58
3.5	Déploiement	58

3.5.1	Conteneurisation Docker	58
3.5.2	Déploiement sur VPS Hetzner	59
3.5.3	Environnements	59
3.6	Évaluation des Performances	59
3.6.1	Méthodologie	59
3.6.2	Temps de Calcul	60
3.6.3	Consommation Mémoire et CPU	61
3.6.4	Qualité des Solutions	62
3.6.5	Comparaison des Services de Routage	63
3.6.6	Scalabilité	63
3.7	Conclusion	64
	Conclusion Générale	65

Liste des figures

2.1	Diagramme de cas d'utilisation du TMS	26
2.2	Diagramme de classes du TMS	27
2.3	Diagramme de séquence : Création d'une commande	29
2.4	Diagramme de séquence : Lancement d'optimisation	30
2.5	Diagramme de séquence : Confirmation de livraison	31
3.1	Interface de création de commande (Expéditeur)	53
3.2	Dashboard principal (Dispatcheur)	54
3.3	Interface d'optimisation avec barre de progression (Dispatcheur)	55
3.4	Visualisation d'une tournée sur carte (Dispatcheur)	55
3.5	Dashboard des tournées (Chauffeur)	56
3.6	Interface d'exécution de tournée (Chauffeur)	57
3.7	Interface d'administration (Administrateur)	58
3.8	Évolution du temps de calcul selon la taille du problème	61
3.9	Consommation mémoire et CPU selon la taille du problème	62
3.10	Qualité des solutions OR-Tools : distance, véhicules, taux de service	62

Liste des tableaux

1.1	Besoins fonctionnels du module Gestion des commandes	18
1.2	Besoins fonctionnels du module Optimisation des tournées	19
1.3	Besoins fonctionnels du module Exécution des tournées	20
1.4	Besoins fonctionnels du module Administration	20
2.1	Paramètres du modèle et leurs valeurs par défaut d'implémentation.	44
2.2	Mapping entre modèle mathématique et API OR-Tools	44
3.1	Temps d'exécution moyen de l'optimisation (moyennes par taille sur 28 exécutions exploitables ; 31 tâches enregistrées au total)	60
3.2	Augmentation mémoire (delta RSS) et utilisation CPU pendant optimisation	61
3.3	Comparaison Google Routes API v2 vs OSRM (100 commandes identiques)	63
3.4	Performance de l'API sous charge (Apache Bench, 0% échecs)	64

Liste des abréviations

Abréviation	Signification
ACID	Atomicité, cohérence, isolation et durabilité
API	Interface de programmation applicative
CI/CD	Intégration continue et déploiement continu
CPU	Processeur central
CRUD	Créer, lire, mettre à jour et supprimer
DBSCAN	Algorithme de clustering basé sur la densité
GLS	Recherche locale guidée (<i>Guided Local Search</i>)
GPS	Système de positionnement global
HTTP	Protocole de transfert hypertexte
IP	Protocole Internet
JSON	Format d'échange de données JavaScript Object Notation
JWT	Jeton web JSON
KPI	Indicateur clé de performance
MILP	Programme linéaire en nombres entiers mixte
MLD	Multi-Level Dijkstra
MT-VRPTW	Problème de tournées de véhicules multi-trajets avec fenêtres de temps
ORM	Mapping objet-relationnel
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
OTD	Livraison à l'heure (<i>On-Time Delivery</i>)
RAM	Mémoire vive
RBAC	Contrôle d'accès basé sur les rôles
REST	Style d'architecture pour les API web
RSS	Mémoire résidente d'un processus
SGBD	Système de gestion de base de données
SPA	Application web monopage
SQL	Langage de requête structuré
SSH	Protocole d'accès distant sécurisé
TMS	Système de gestion du transport
UML	Langage de modélisation unifié
URL	Adresse d'une ressource web
VPS	Serveur privé virtuel
VRP	Problème de tournées de véhicules
VRPTW	Problème de tournées de véhicules avec fenêtres de temps

Introduction Générale

Contexte général

La logistique du dernier kilomètre — l'étape finale d'acheminement depuis un centre de distribution jusqu'au client — représente une part importante des coûts logistiques. Cette proportion élevée s'explique par la difficulté de l'opération : grande dispersion géographique des clients, fenêtres de temps strictes, véhicules de capacités et de types différents (poids, volume, température), et la nécessité d'optimiser plusieurs objectifs en même temps (distance, nombre de véhicules, taux de service).

Problématique

La planification optimale des tournées de livraison constitue un défi mathématique et informatique majeur. D'un point de vue mathématique, cela correspond au *Vehicle Routing Problem with Time Windows* (VRPTW), un problème d'optimisation combinatoire NP-difficile dont l'espace de solutions croît de manière factorielle ($O(n!)$). Pour 50 clients, il existe plus de 3×10^{64} arrangements possibles, ce qui rend la recherche de la solution parfaite par force brute impossible.

Les approches manuelles, encore largement utilisées par les entreprises de transport mauritaniennes, atteignent vite leurs limites face à ce volume de calculs. Il en résulte des solutions sous-optimales qui génèrent des surcoûts opérationnels (distances excessives, véhicules sous-utilisés) et des retards de livraison.

Objectifs du projet

Ce projet de fin d'études vise à concevoir et développer un **Système de Gestion du Transport (TMS)** intégrant un moteur d'optimisation mathématique pour résoudre le VRPTW dans le contexte mauritanien. Les objectifs se déclinent en trois axes complémentaires :

Axe algorithmique et mathématique : Modéliser formellement le problème VRPTW avec ses contraintes (multi-trajets, fenêtres de temps, types de véhicules, capacités mul-

tiples) et implémenter une méthode de résolution par métaheuristique.

Axe technique : Développer une application web complète avec une architecture distribuée (backend FastAPI, worker Celery pour l’optimisation asynchrone, frontend React), et l’intégrer avec des services de cartographie (Google Geocoding API pour la géolocalisation, avec repli Nominatim, et OSRM auto-hébergé pour le routage).

Axe fonctionnel : Répondre aux besoins concrets de l’entreprise de transport en proposant des interfaces adaptées à chaque utilisateur (expéditeur, dispatcheur, chauffeur, administrateur), avec un suivi en temps réel et des tableaux de bord.

Domaines d’application

La réalisation de ce projet s’appuie sur plusieurs domaines de l’informatique et des mathématiques :

- **Recherche opérationnelle :** Modélisation mathématique du VRPTW et implémentation d’un solveur basé sur Google OR-Tools.
- **Génie logiciel :** Architecture distribuée avec exécution asynchrone (Celery), contrôle d’accès par rôle (RBAC), et conteneurisation (Docker).
- **Ingénierie réseau :** Évaluation comparative de deux approches (Google Routes API avec chunking vs OSRM avec requête unique), conduisant au choix d’OSRM pour réduire fortement la latence réseau et éviter les limitations d’API.
- **Déploiement système :** Mise en production sur serveur VPS (Hetzner) avec base de données PostgreSQL et serveur de routage OSRM auto-hébergé.

Méthodologie

Le développement du système a suivi une approche itérative : spécification, développement (backend + frontend), tests locaux, déploiement en production via GitHub Actions. Le code source est versionné avec Git.

Organisation du rapport

Ce rapport est structuré en trois chapitres principaux :

- **Chapitre 1 — Analyse des besoins et étude préalable :** Présente la problématique de la livraison, la solution proposée, et identifie les besoins fonctionnels et non fonctionnels du système.

- **Chapitre 2 — Conception globale et modélisation** : Détaille la conception logicielle (diagrammes UML) et la modélisation mathématique du problème (équations du VRPTW).
- **Chapitre 3 — Réalisation et évaluation** : Décrit les technologies utilisées (FastAPI, React, OR-Tools), l'implémentation du solveur, l'interface utilisateur, et évalue les performances du système (temps de calcul, consommation CPU/RAM).

Une conclusion générale synthétise les résultats obtenus et propose des perspectives d'évolution.

Chapitre 1

Analyse des besoins et Étude préalable

1.1 Introduction

Ce chapitre ancre le projet dans son contexte opérationnel spécifique : une entreprise de transport mauritanienne confrontée aux défis quotidiens de planification de tournées. Après une analyse détaillée de la problématique terrain (section 1.2), nous établissons les besoins fonctionnels et non fonctionnels du système (sections 1.3 et 1.4), qui serviront de cahier des charges pour la conception présentée au chapitre suivant.

1.2 Présentation de la problématique

1.2.1 La complexité de la livraison du dernier kilomètre

La livraison du dernier kilomètre désigne l'étape finale du processus logistique : acheminer les marchandises depuis un centre de distribution jusqu'au client final. Cette étape représente une part importante des coûts logistiques et influence fortement la satisfaction client (respect des délais, qualité de la livraison).

Les défis sont multiples :

Dispersion géographique. Dans les zones urbaines mauritaniennes comme Nouakchott, les clients sont dispersés avec des densités variables. Le centre-ville commercial présente une forte concentration de commandes, tandis que les zones résidentielles périphériques nécessitent de longs déplacements pour peu de clients.

Contraintes temporelles strictes. Les fenêtres de temps sont précises et réduisent considérablement l'espace de solutions réalisables. Un restaurant doit être livré avant l'heure du service, une pharmacie souhaite recevoir ses médicaments en début de matinée.

Hétérogénéité des demandes. Les commandes varient fortement en poids (quelques kilogrammes à plusieurs centaines), volume, et nature (produits généraux, réfrigérés, surgelés). Cette hétérogénéité impose une flotte de véhicules diversifiée.

Optimisation multi-critères. L'entreprise doit simultanément minimiser des objectifs conflictuels : distance parcourue, nombre de véhicules utilisés, respect des fenêtres de temps, maximisation du taux de remplissage.

1.2.2 Limites des approches manuelles

La planification manuelle des tournées par les dispatcheurs présente plusieurs limites critiques :

Incapacité à explorer l'espace de solutions. L'espace de recherche croît de manière factorielle $O(n!)$. Pour $n = 50$ clients, le nombre d'arrangements dépasse 3×10^{64} , rendant toute approche par force brute mathématiquement impossible. Aucun humain ne peut examiner qu'une infime fraction de cet espace, ce qui conduit quasi-systématiquement à des solutions sous-optimales.

Biais cognitifs. Les dispatcheurs appliquent des règles heuristiques simples (regrouper par quartier) qui, bien que raisonnables, ignorent les interactions complexes entre contraintes. Une tournée géographiquement compacte peut violer les fenêtres de temps si les clients du même quartier ont des horaires incompatibles.

Absence de reproductibilité. Deux dispatcheurs produisent des plans différents pour le même ensemble de commandes. Les performances fluctuent selon la charge cognitive (fatigue, interruptions).

Difficulté à intégrer de nouvelles contraintes. L'ajout d'une contrainte supplémentaire (interdire certaines combinaisons commande-véhicule) complique exponentiellement la tâche manuelle.

1.2.3 Contexte opérationnel mauritanien

Notre projet s'inscrit dans le contexte d'une entreprise de transport mauritanienne opérant dans deux villes principales :

- **Nouakchott** : Capitale avec un réseau routier partiellement structuré, forte concentration de commerces et demande de livraison élevée.
- **Nouadhibou** : Ville portuaire au nord avec contraintes d'accès spécifiques (zones portuaires, industries de pêche).

Caractéristiques de la flotte. Le scénario de test considère une entreprise disposant des véhicules répartis en trois catégories :

- **Véhicules standards** : Camionnettes de capacité 800-1000 kg et 8-10 m³, pour marchandises générales.
- **Véhicules réfrigérés** : Systèmes de réfrigération maintenant 0-5°C, pour produits périssables.
- **Véhicules à congélation** : Températures négatives (-18°C), pour produits surgelés.

Profil des commandes. Le scénario représentatif considère 40 à 100 commandes quotidiennes . Clients principaux :

- **Pharmacies** : Livraisons matinales (08h00-10h00), fenêtres strictes, souvent produits réfrigérés.
- **Restaurants et hôtels** : Livraisons avant service, fenêtres de temps généralement respectées avec tolérance limitée.
- **Supermarchés et commerces** : Horaires flexibles (09h00-17h00), gros volumes.
- **Particuliers et bureaux** : Fenêtres larges, petits volumes.

1.2.4 Formulation du problème : VRPTW multi-contraint

Le problème se formalise comme une extension du *Vehicle Routing Problem with Time Windows* (VRPTW) classique, avec contraintes supplémentaires :

- **Capacités multiples** : Chaque véhicule a deux capacités à respecter simultanément (poids et volume).
- **Fenêtres de temps strictes** : Chaque commande impose une fenêtre de temps stricte (aucune livraison hors fenêtre). Le modèle de données et le solveur prévoient également des fenêtres souples avec pénalité (champ `time_window_type` : HARD/SOFT), mais l'API ne l'expose pas encore ; en production, toutes les commandes utilisent donc le mode strict (HARD) par défaut.
- **Types de véhicules** : Certaines commandes requièrent un type spécifique (produits surgelés → véhicule congélateur).
- **Multi-trajets** : Un véhicule peut effectuer plusieurs tournées dans la journée en retournant au dépôt pour se recharger.
- **Multi-dépôts** : Le système doit gérer plusieurs entrepôts dans différentes villes (chaque optimisation traite un dépôt à la fois).

Le VRPTW est un problème NP-difficile nécessitant une approche algorithmique avancée pour trouver des solutions de bonne qualité en temps raisonnable (environ 2.5-4.5 min pour 50-100+ commandes dans les tests réalisés).

1.3 Présentation de la solution proposée

1.3.1 Vision générale

Nous proposons un **Système de Gestion du Transport (TMS)** intégrant un moteur d'optimisation mathématique. La solution comporte :

Optimisation mathématique. Un solveur utilisant la métaheuristique *Guided Local Search* (GLS) via OR-Tools, motivée par la NP-difficulté du VRPTW. Le solveur obtient un taux de service $> 85\%$ en 30s-4.5min lorsque la flotte et les chauffeurs disponibles sont suffisants ; les scénarios volontairement contraints mesurent plutôt la robustesse face au manque de ressources.

Application web. Gestion du cycle de vie des commandes (création, optimisation, affectation, suivi, livraison) avec contrôle d'accès par rôle (expéditeur, dispatcheur, chauffeur, administrateur) et interfaces spécifiques à chaque acteur.

Infrastructure technique. Architecture séparant l'API REST (réponse immédiate) du calcul d'optimisation (worker asynchrone), base de données PostgreSQL, service de routage OSRM (données OpenStreetMap Mauritanie), frontend React, et déploiement VPS avec pipeline GitHub Actions.

1.3.2 Architecture globale

Le système adopte une architecture en trois couches :

Couche Présentation. Application web monopage offrant des interfaces différenciées par rôle :

- **Expéditeur** : Créer et suivre ses commandes.
- **Dispatcheur** : Lancer les optimisations, gérer les tournées et anomalies.
- **Chauffeur** : Consulter sa tournée et confirmer les livraisons.
- **Administrateur** : Gérer utilisateurs, flotte, entrepôts et KPI.

Couche Métier. Serveur HTTP exposant une API REST documentée, gérant :

- **Authentification et autorisation** : Tokens cryptographiques, contrôle d'accès par rôle.
- **Opérations CRUD** : Gestion des entités métier.
- **Orchestration de l'optimisation** : Création de tâches, délégation au worker, suivi de progression.
- **Logique métier** : Validation des contraintes, gestion des statuts.

Couche Optimisation. Processus d'arrière-plan exécutant les tâches d'optimisation de manière asynchrone :

- Récupération des données depuis la base de données.
- Construction de la matrice de distances et temps réels via un service de routage.
- Construction du modèle de routage avec toutes les contraintes.
- Résolution avec métaheuristique et limites de temps dynamiques (30/60/120s selon la taille).
- Persistance des résultats (création des tournées, affectation des commandes).

Services externes. Le système s'appuie sur des services de routage et de géolocalisation pour calculer les distances réelles sur le réseau routier mauritanien et convertir les adresses en coordonnées GPS.

Optimisation réseau critique. La matrice complète de distances et temps est récupérée en une seule requête HTTP groupée. Sans cette optimisation, le système serait inutilisable en production.

1.3.3 Flux fonctionnel principal

Le flux complet se déroule en sept étapes :

1. **Création de commandes :** Les expéditeurs créent des commandes via l'interface web (adresse, poids, volume, date, fenêtres horaires, type de véhicule). Le système géolocalise automatiquement l'adresse et valide les contraintes.
2. **Lancement de l'optimisation :** Le dispatcheur sélectionne un entrepôt et une date, puis lance l'optimisation. Le système propose toutes les commandes en attente pour cette date et tous les véhicules disponibles.
3. **Calcul asynchrone :** Le backend crée une tâche d'optimisation et la délègue au worker. Le frontend affiche une barre de progression et interroge périodiquement le statut (toutes les 3 secondes). L'utilisateur peut naviguer ailleurs pendant le calcul.
4. **Construction de la matrice :** Le worker appelle le service de routage pour obtenir la matrice complète de distances et temps réels.
5. **Résolution :** Le worker construit le modèle de routage avec dimensions de capacité et de temps, puis lance la métaheuristique avec limites dynamiques (30/60/120s selon la taille du problème).
6. **Création des tournées :** À la fin, le worker crée les enregistrements de tournées dans la base de données, affecte automatiquement un chauffeur à chaque tournée, et met à jour le statut des commandes (affectée ou non affectée).

7. **Exécution et suivi** : Les chauffeurs consultent leur tournée, démarrent la tournée, confirment chaque livraison (enregistrement de l'heure réelle), et terminent la tournée. Le dispatcheur suit la progression en temps réel sur une carte interactive.

Avantages observés :

- Optimisation automatique des distances parcourues.
- Maximisation du taux de service des commandes.
- Temps de planification drastiquement réduit (ordre de minutes au lieu d'heures).
- Meilleure utilisation de la capacité des véhicules.

1.4 Identification des besoins fonctionnels

Les besoins fonctionnels décrivent **ce que le système doit faire**. Le tableau suivant synthétise l'ensemble des besoins identifiés, organisés par module fonctionnel.

1.4.1 Synthèse des besoins fonctionnels

ID	Description résumée	Acteurs
BF-1	Créer une commande avec adresse, poids, volume, date, fenêtres horaires, type de véhicule. Géolocalisation automatique.	Expéditeur, Administrateur
BF-2	Consulter la liste des commandes avec filtrage par statut, date, entrepôt. Afficher ID, expéditeur, adresse, poids, volume, date, fenêtres, statut, véhicule et chauffeur affectés.	Expéditeur (propres), Dispatcheur (toutes), Administrateur (toutes), Chauffeur (affectées)
BF-3	Modifier une commande. Règles : EN_ATTENTE modifiable librement ; AFFECTEE modifiable mais remise en EN_ATTENTE et retirée de la tournée (avec avertissement) ; EN_COURS/LIVREE/ANNULEE non modifiable ; NON_AFFECTEE modifiable pour correction.	Expéditeur (propres EN_ATTENTE), Administrateur (toutes)
BF-4	Annuler une commande (statut ANNULEE). Si affectée, retirer de la tournée. Contrainte : commande LIVREE non annulable.	Expéditeur (propres non livrées), Administrateur (toutes)

TABLE 1.1 – Besoins fonctionnels du module Gestion des commandes

Module Gestion des Commandes

ID	Description résumée	Acteurs
BF-5	Lancer le calcul d'optimisation pour générer les tournées. Paramètres : entrepôt, date, commandes (défaut : EN_ATTENTE pour la date), véhicules (défaut : DISPONIBLE de l'entrepôt). Processus asynchrone (30s-4.5min selon taille), création de tournées et affectation des commandes (AFFECTEE ou NON_AFFECTEE).	Dispatcheur, Administrateur
BF-6	Suivre en temps réel la progression de l'optimisation (pourcentage 0-100%, statut EN_ATTENTE/EN_COURS/TERMINEE/ERREUR). Interrogation périodique toutes les 3 secondes.	Dispatcheur, Administrateur
BF-7	Consulter les résultats d'optimisation : métriques globales (distance totale, nombre de véhicules utilisés, commandes affectées/non servies, temps d'exécution), liste détaillée des tournées avec séquence des livraisons, visualisation cartographique interactive, liste des commandes non servies avec raisons.	Dispatcheur, Administrateur
BF-8	Consulter l'historique des optimisations (tri par date décroissante) avec date d'exécution, algorithme, statut, métriques. Utilité : analyser l'évolution des performances, comparer configurations.	Dispatcheur, Administrateur

TABLE 1.2 – Besoins fonctionnels du module Optimisation des tournées

Module Optimisation des Tournées

ID	Description résumée	Acteurs
BF-9	Visualiser sa tournée du jour avec véhicule affecté, liste ordonnée des arrêts (adresse, poids, volume, heure prévue, fenêtre), carte interactive (dépôt, points de livraison, itinéraire), statut de la tournée (PLANIFIEE/EN_COURS/TERMINEE).	Chauffeur
BF-10	Démarrer une tournée : enregistrer l'heure de départ, changer les statuts (tournée → EN_COURS, chauffeur/véhicule → EN_MISSION, commandes → EN_COURS). Contrainte : date de la tournée = date du jour.	Chauffeur
BF-11	Confirmer une livraison à chaque arrêt : enregistrer l'heure réelle, changer les statuts (arrêt → LIVREE, commande → LIVREE), mettre à jour la progression de la tournée (% = arrêts livrés / arrêts non annulés × 100).	Chauffeur
BF-12	Signaler une anomalie (retard, colis endommagé, absence client, autre) associée à un arrêt ou à la tournée globale. Type d'anomalie et description textuelle. Notification au dispatcheur.	Chauffeur
BF-13	Terminer une tournée lorsque tous les arrêts non annulés sont livrés : changer les statuts (tournée → TERMINEE, chauffeur/véhicule → DISPONIBLE), enregistrer l'heure de retour au dépôt. Contrainte : bouton visible uniquement si progression = 100%.	Chauffeur

TABLE 1.3 – Besoins fonctionnels du module Exécution des tournées

Module Exécution des Tournées

ID	Description résumée	Acteurs
BF-14	Gérer les utilisateurs : créer, modifier, activer/désactiver. Informations : nom, email, téléphone, rôle (EXPEDITEUR/DISPATCHEUR/CHAUFFEUR/ADMINISTRATEUR), mot de passe haché, entrepôt de rattachement.	Administrateur
BF-15	Gérer la flotte : créer, modifier, supprimer des véhicules. Informations : immatriculation unique, capacité poids/volume, type (NORMAL/REFRIGERE/CONGELATEUR), statut (DISPONIBLE/EN_MISSION/HORS_SERVICE), entrepôt de rattachement. Contrainte : véhicule non supprimable si référencé par une tournée.	Administrateur
BF-16	Gérer les entrepôts : créer, modifier, activer/désactiver. Informations : nom, ville, adresse, coordonnées GPS, horaires d'ouverture/fermeture. Utilité : gérer plusieurs dépôts (Nouakchott, Nouadhibou).	Administrateur
BF-17	Consulter les indicateurs de performance (KPI) : taux de livraison à l'heure (OTD), coût par kilomètre (distance / commandes livrées), taux d'utilisation de la flotte, nombre de commandes non servies, évolution temporelle (graphiques 30 derniers jours).	Dispatcheur, Administrateur

TABLE 1.4 – Besoins fonctionnels du module Administration

1.5 Identification des besoins non fonctionnels

Les besoins non fonctionnels définissent **comment le système doit fonctionner** en termes de qualités globales. Nous les organisons en quatre grandes catégories.

1.5.1 Performances

- **Temps de réponse API** : Les opérations de lecture (consultation de commandes, tournées, véhicules) doivent avoir un temps de réponse < 500 ms dans 95% des cas, garantissant une expérience utilisateur fluide. Moyens : indexation appropriée des colonnes fréquemment filtrées, requêtes optimisées.
- **Temps de calcul de l'optimisation** : Exécution en 30s-4.5min selon la taille du problème. Moyens : métaheuristique GLS, limites de temps configurables (30/60/120s selon la taille).
- **Optimisation des requêtes réseau** : La construction de la matrice de distances doit utiliser des appels HTTP groupés pour éviter la saturation réseau. Moyens : batching des requêtes API (détaillé en section 3.2).
- **Réactivité de l'interface** : L'interface ne doit jamais se bloquer pendant les opérations longues. Moyens : optimisation asynchrone, affichage immédiat de barre de progression, chargement paresseux avec pagination (limite 100 éléments/page).

1.5.2 Fiabilité et disponibilité

- **Disponibilité du service** : 99% du temps (tolérance 7h d'indisponibilité/mois). L'entreprise opère 6 jours/semaine ; une indisponibilité prolongée bloquerait la planification. Moyens : conteneurs configurés pour redémarrage automatique, monitoring avec logs centralisés, sauvegardes quotidiennes.
- **Tolérance aux pannes** : Si l'appel groupé au service de routage échoue (requête bulk /table timeout), le système bascule automatiquement sur une boucle $N \times N$ de requêtes individuelles au même service. Ce fallback dégrade les performances (latence $\times N^2$) mais permet de compléter l'optimisation si les requêtes individuelles aboutissent.
- **Gestion des erreurs d'optimisation** : Si une tâche échoue (contraintes impossibles, timeout, erreur interne), le système doit marquer la tâche en ERREUR, stocker un message explicite, et afficher des suggestions de correction ("Aucune solution trouvée. Vérifiez les fenêtres de temps ou ajoutez des véhicules.").

1.5.3 Sécurité

- **Authentification sécurisée** : Seuls les utilisateurs authentifiés peuvent accéder aux fonctionnalités. Moyens : hachage des mots de passe avec algorithme résistant aux attaques par force brute, authentification via tokens cryptographiques avec expiration (24h), transmission via headers HTTP sécurisés, validation automatique à chaque requête.
- **Contrôle d'accès par rôle (RBAC)** : Chaque utilisateur n'a accès qu'aux fonctionnalités autorisées. Exemples : EXPEDITEUR consulte uniquement ses commandes (filtrage côté serveur) ; seuls DISPATCHEUR et ADMINISTRATEUR lancent des optimisations ; seuls CHAUFFEUR démarrent/terminent des tournées ; seuls ADMINISTRATEUR gèrent les utilisateurs et la flotte. Moyens : vérification du rôle côté serveur, réponse HTTP 403 Forbidden si non autorisé.
- **Protection des données sensibles** : Les bases de données ne doivent **pas être accessibles publiquement** depuis Internet. Moyens : aucun mapping de ports publics, communication interne uniquement via réseau privé, seuls les points d'entrée applicatifs exposés.

1.5.4 Utilisabilité et maintenabilité

- **Interfaces adaptées par rôle** : Chaque acteur dispose d'une interface adaptée à ses besoins sans être submergé. Interface CHAUFFEUR centrée sur la tournée du jour avec carte interactive ; interface EXPEDITEUR centrée sur création et suivi de commandes ; interface DISPATCHEUR avec vue globale, optimisations, et KPI.
- **Visualisation cartographique** : Les tournées doivent être visualisables sur une carte interactive avec itinéraire tracé et marqueurs pour chaque point de livraison. Marqueurs color-codés par statut.
- **Modularité du code** : Le code doit être structuré en modules clairement séparés pour faciliter la maintenance et l'évolution. Séparation en couches (modèles de données, schémas de validation, routeurs API, services métier, solveurs), composants réutilisables, documentation complète.
- **Architecture Celery** : Le système utilise Celery pour découpler l'API du calcul. Redis sert de broker pour distribuer les tâches aux workers disponibles.
- **Séparation des environnements** : Configurations de développement et de production séparées pour éviter les erreurs de déploiement. Fichiers de configuration distincts, stacks Docker distincts, variables d'environnement injectées au build.
- **Conteneurisation et portabilité** : Le système doit s'exécuter sur différentes plateformes (macOS arm64, Linux x86_64) sans modification. Toutes les dépendances conteneurisées, images multi-architectures.

- **Déploiement reproductible** : Le déploiement sur un nouveau serveur doit être simple et reproductible. Moyens : scripts de déploiement automatisés, pipeline d'intégration/déploiement continu, documentation complète, import automatique de la base de données au premier lancement.

1.6 Conclusion

Ce chapitre a analysé la problématique de la livraison du dernier kilomètre dans le contexte mauritanien, en mettant en évidence la complexité du VRPTW multi-contraint et les limites des approches manuelles. L'espace de recherche croissant de manière factorielle ($O(n!)$ pour n clients) rend toute exploration manuelle inefficace et conduit à des solutions systématiquement sous-optimales.

Nous avons présenté la solution proposée : un système permettant de créer des commandes, de lancer l'optimisation des tournées via un solveur OR-Tools, et de suivre l'exécution des livraisons. L'architecture sépare l'API web (réponse immédiate) du calcul d'optimisation (exécuté en arrière-plan par un worker Celery).

L'identification des besoins fonctionnels (17 besoins organisés en 4 modules) et non fonctionnels fournit un cahier des charges. Les contraintes de performance identifiées incluent : la réduction du nombre d'appels réseau pour la construction de la matrice de distances (requête unique OSRM au lieu de N^2 appels individuels), et des limites de temps de calcul dynamiques selon la taille du problème (30/60/120s pour petites/moyennes/grandes instances).

Le chapitre suivant détaillera la conception globale du système sous deux angles complémentaires : conception logicielle (diagrammes UML) et conception mathématique (modèle VRPTW formel avec paramètres, variables, fonction objectif, et contraintes).

Chapitre 2

Conception Globale et Modélisation

2.1 Introduction

Le chapitre précédent a établi le cahier des charges du système à partir d'une analyse approfondie des besoins métier. Ce deuxième chapitre présente la conception globale de la solution selon deux perspectives complémentaires et indissociables.

La **conception logicielle** (section 2.1) formalise l'architecture du système via des diagrammes UML (Unified Modeling Language). Les diagrammes de cas d'utilisation identifient les interactions entre acteurs et système ; le diagramme de classes structure le modèle de données ; et les diagrammes de séquence illustrent les flux critiques (création de commande, lancement d'optimisation, confirmation de livraison). Cette modélisation oriente les choix technologiques et garantit la cohérence entre interfaces, services métier, et couche de persistance.

La **conception mathématique** (section 2.2) formalise le problème d'optimisation comme un programme linéaire en nombres entiers (MILP). Le modèle VRPTW définit les paramètres (commandes, véhicules, distances, temps), les variables de décision (affectations client-véhicule, ordre de visite), la fonction objectif (minimiser distance totale et nombre de véhicules), et les contraintes (capacités multiples, fenêtres de temps souples/dures, types de véhicules, multi-trajets).

Ce formalisme mathématique permet de choisir une méthode de résolution adaptée (métaheuristique OR-Tools avec Recherche Locale Guidée).

Ces deux conceptions convergent vers une seule implémentation : le code traduit les contraintes mathématiques en appels au solveur OR-Tools, et les classes logicielles (Commande, Vehicule, Tournee) correspondent aux principales entités du modèle mathématique.

2.2 Conception Logicielle (UML)

2.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 2.1) identifie les quatre acteurs principaux et leurs interactions avec le système :

Expéditeur : Création et suivi de commandes (géolocalisation automatique, annulation si EN_ATTENTE).

Dispatcheur : Orchestration des opérations (lancement d'optimisations asynchrones, consultation de l'historique, gestion des tournées et anomalies, KPI).

Chauffeur : Exécution des tournées (démarrage, confirmation de livraisons avec horodatage, signalement d'anomalies, navigation GPS).

Administrateur : Gestion du système (utilisateurs avec RBAC, flotte avec capacités/types, entrepôts multi-sites) + accès Dispatcheur.

Relations clés : « Lancer une optimisation » inclut « Construire matrice de distances » et « Résoudre VRPTW ». « Confirmer livraison » étend « Mettre à jour progression ».

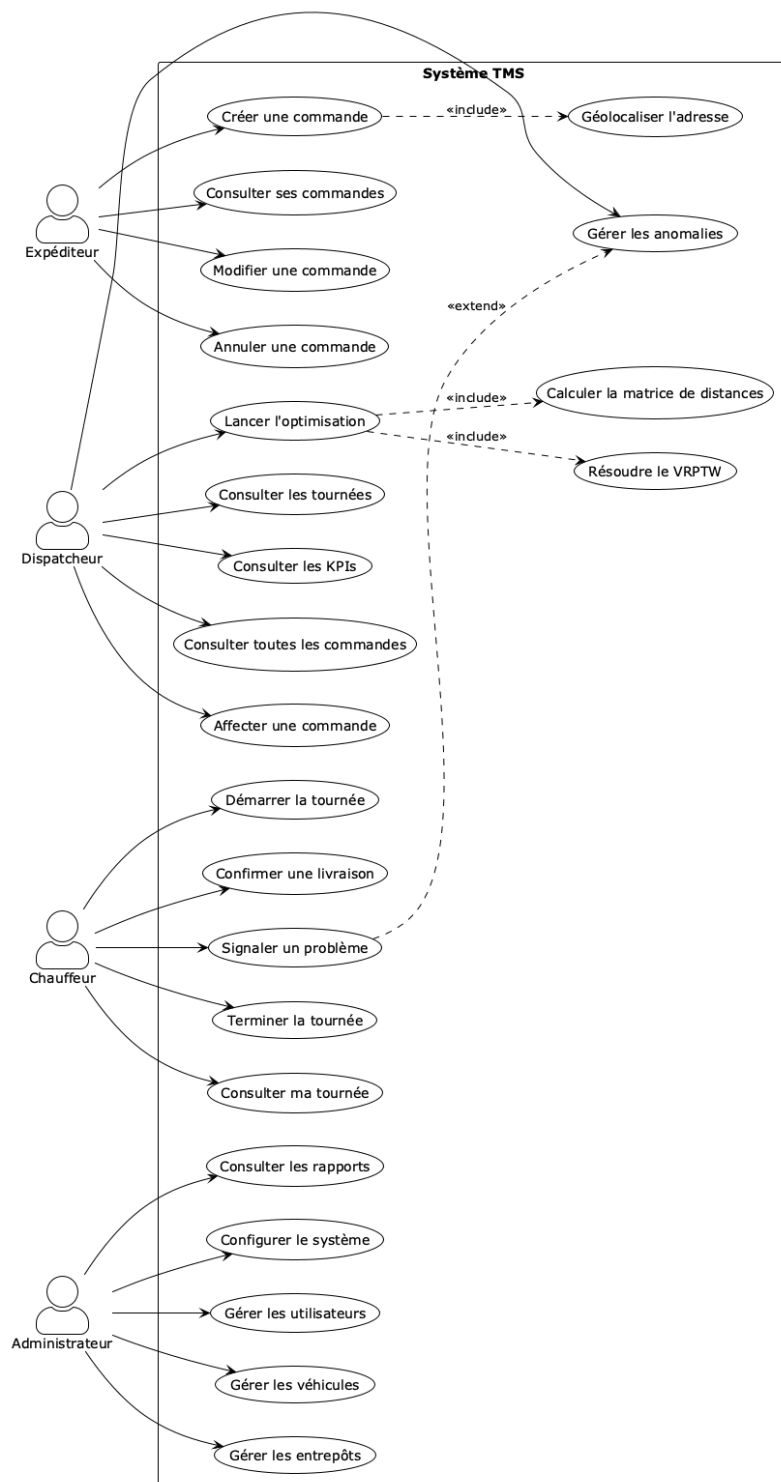


FIGURE 2.1 – Diagramme de cas d'utilisation du TMS

2.2.2 Diagramme de classes

Le diagramme de classes (figure 2.2) structure le modèle de données avec 8 entités : **Utilisateur** (rôles RBAC), **Warehouse** (dépôts multi-sites), **Vehicule** (capacités poids/volume + types), **Commande** (fenêtres de temps), **Tournee** (itinéraires), **Stop-Tournee** (arrêts), **Anomalie** (incidents), et **TacheOptimisation** (traçabilité).

Les capacités doubles (poids ET volume) imposent deux contraintes simultanées dans le VRPTW. Le type de véhicule filtre les commandes compatibles (REFRIGERE/CONGELATEUR). Les clés étrangères préservent l'historique (pas de suppression en cascade).

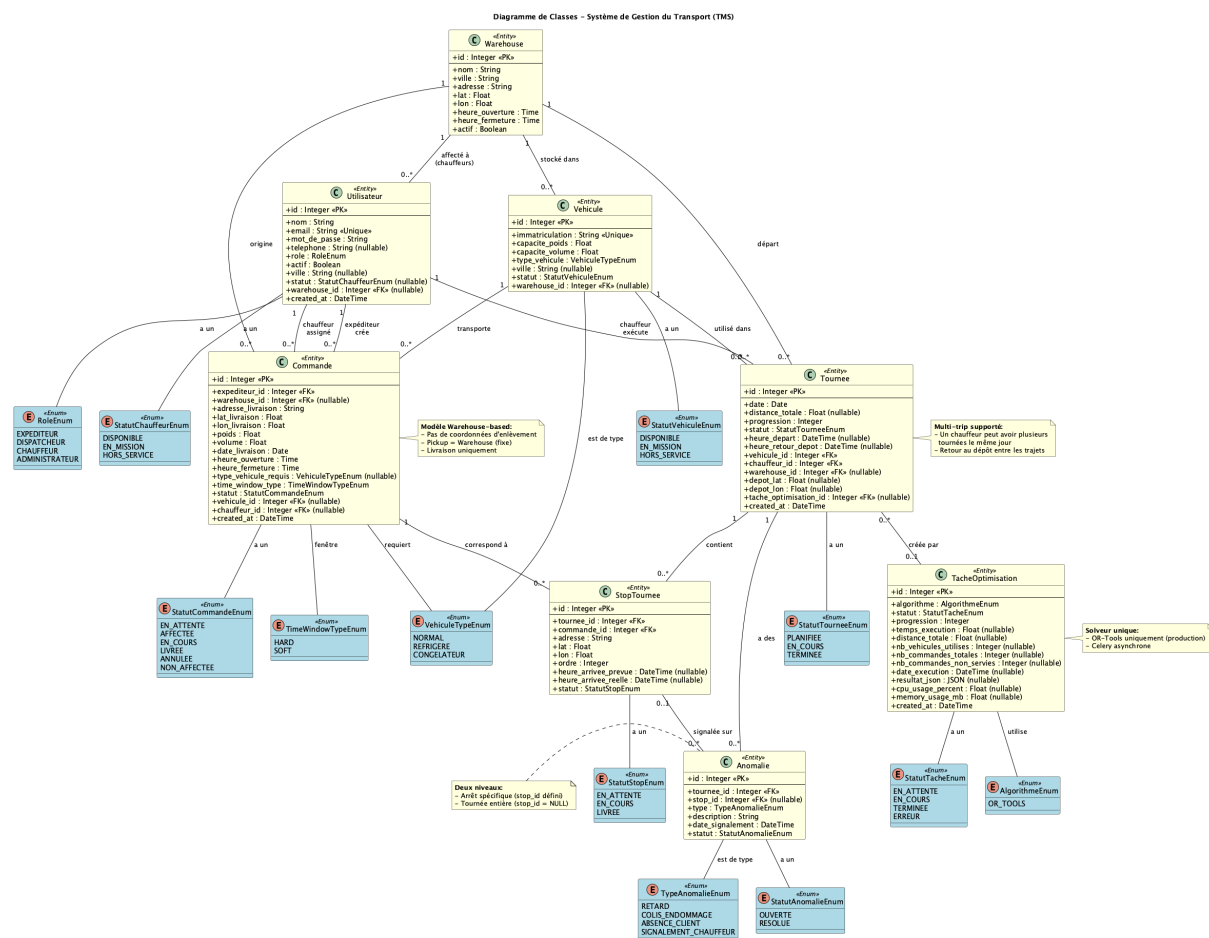


FIGURE 2.2 – Diagramme de classes du TMS

2.2.3 Diagrammes de séquence

Les diagrammes de séquence (figures 2.3, 2.4, 2.5) présentent trois scénarios principaux du système.

Scénario 1 : Création d’une commande. Le géocodage est effectué côté frontend avant la soumission. L’expéditeur saisit une adresse, le frontend interroge d’abord Google Geocoding API, puis Nominatim (OpenStreetMap) en fallback si Google échoue. Les

coordonnées (lat, lon) sont stockées dans le formulaire et affichées à l'utilisateur. **Point clé :** Si les deux services échouent, la validation côté client bloque la soumission du formulaire. Le backend reçoit une requête POST avec lat/lon déjà renseignés, valide la date de livraison ($\geq$ demain), puis enregistre la commande.

Scénario 2 : Lancement d'une optimisation (flux asynchrone). Le flux implique le Dispatcheur, l'API Backend, Redis (broker), un Worker Celery, le solveur OR-Tools, le service de routage (OSRM pour le calcul des matrices de distances/temps), et la base de données.

Le workflow est asynchrone : la requête HTTP retourne immédiatement ($< 1s$) avec un `task_id`, tandis que le calcul (30s-4.5min selon taille) s'exécute en arrière-plan. L'API et le Worker communiquent via Redis. Le Frontend interroge `GET /api/v1/optimisation/{id}/statut` toutes les 3 secondes pour suivre la progression. La matrice de distances complète est obtenue via requête unique à OSRM.

Scénario 3 : Confirmation de livraison. Le flux implique le Chauffeur, l'API Backend, et la base de données. **Point clé :** L'attribut `heure_arrivee_reelle` permet d'analyser a posteriori les écarts entre prévisions et réalité terrain (KPI : taux de respect des fenêtres).

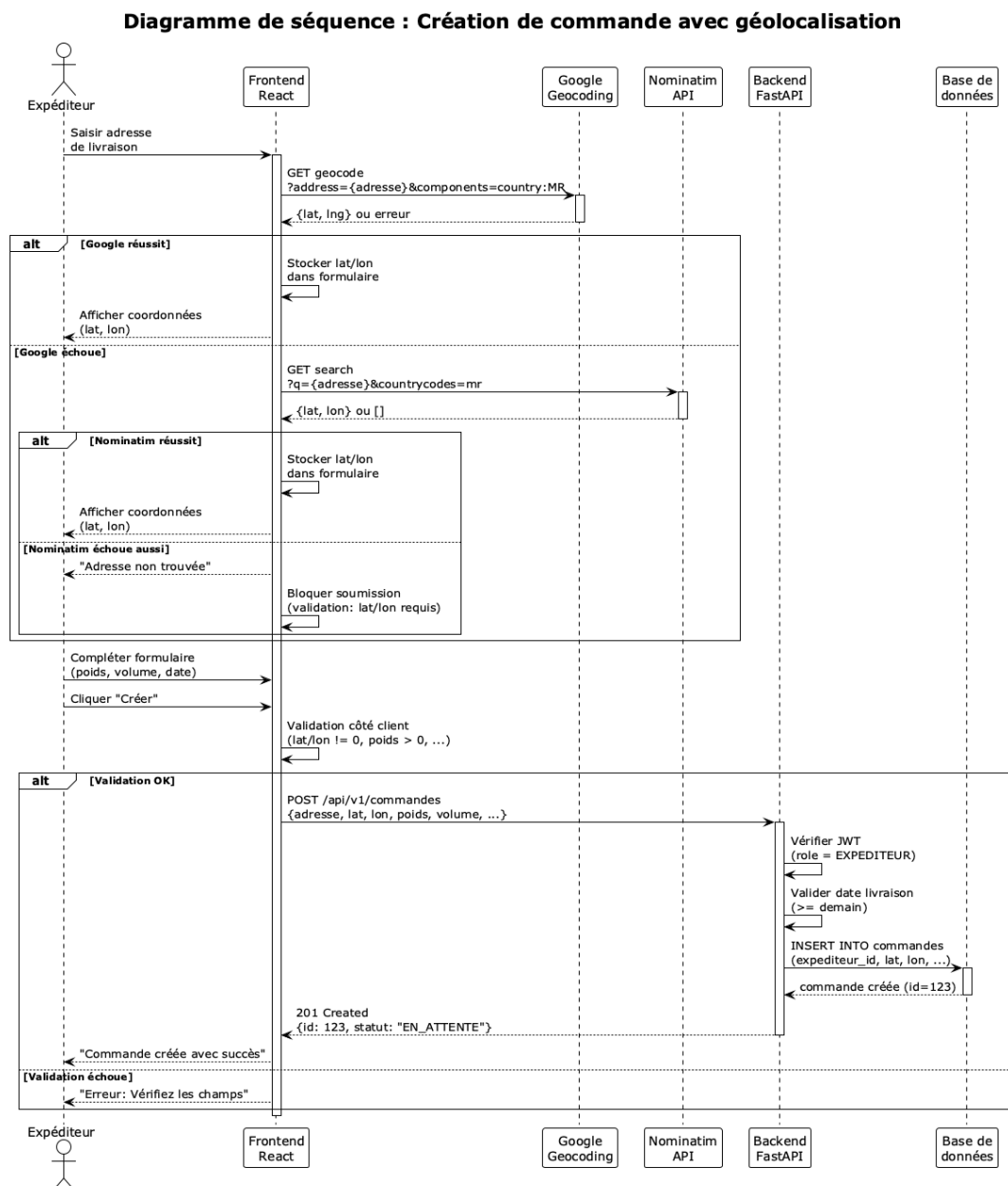


FIGURE 2.3 – Diagramme de séquence : Création d'une commande

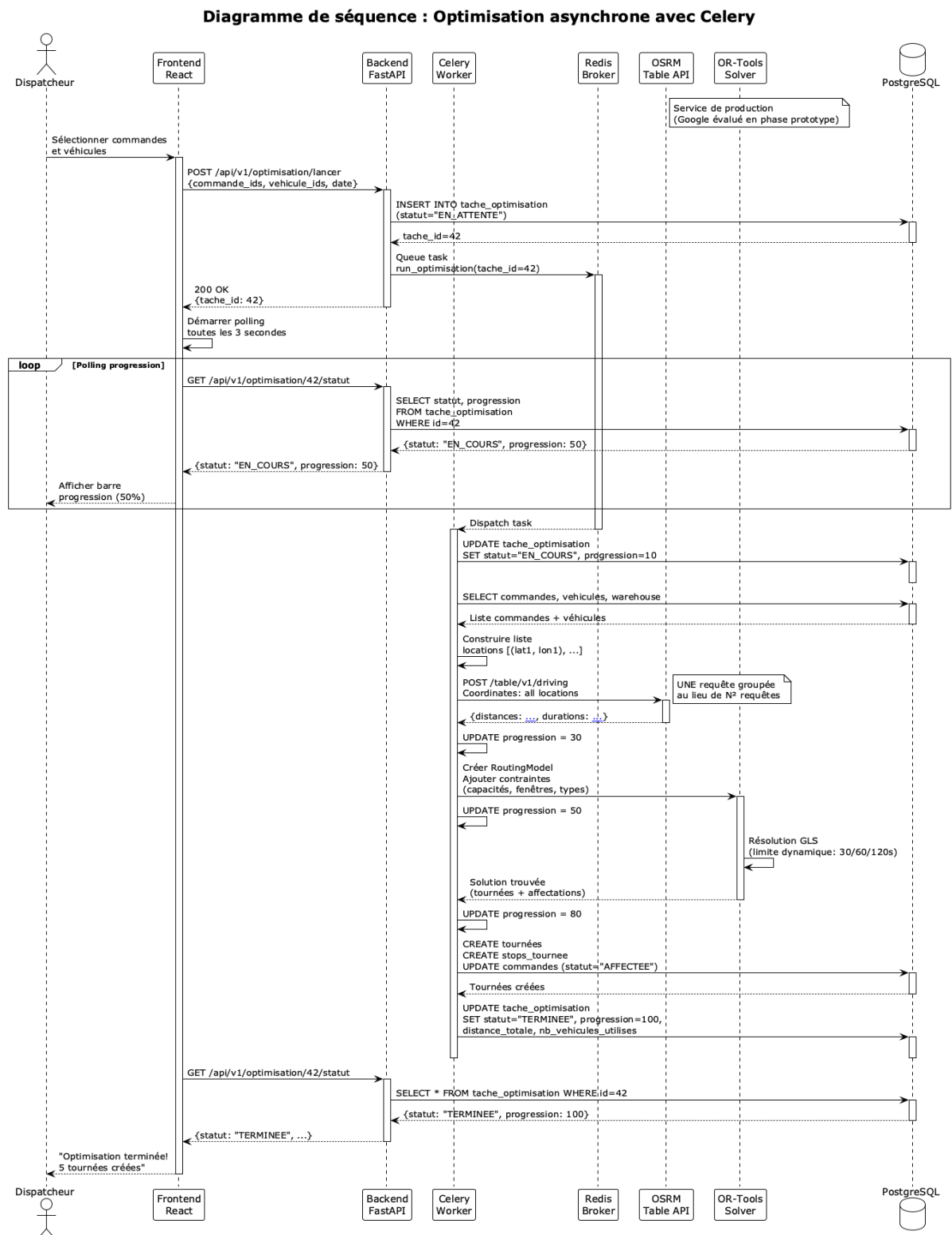


FIGURE 2.4 – Diagramme de séquence : Lancement d'optimisation

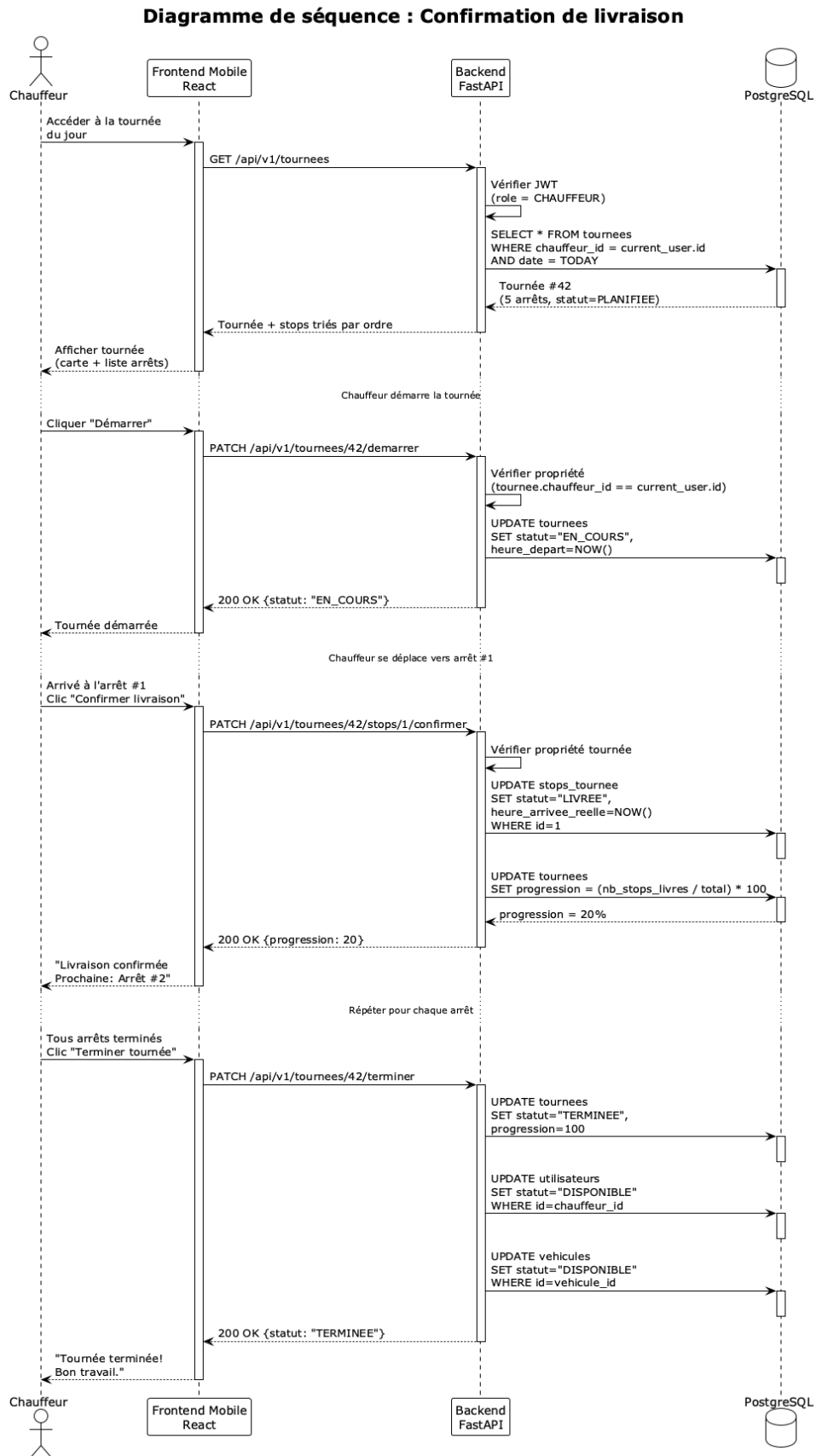


FIGURE 2.5 – Diagramme de séquence : Confirmation de livraison

2.3 Conception Mathématique : Le Modèle VRPTW

Cette section formalise le problème d’optimisation des tournées comme un programme linéaire en nombres entiers (MILP — Mixed Integer Linear Program). Le modèle VRPTW (Vehicle Routing Problem with Time Windows) que nous présentons intègre plusieurs extensions pour refléter les contraintes opérationnelles identifiées au chapitre 1 : capacités multiples (poids et volume), types de véhicules hétérogènes, fenêtres de temps souples et dures, et support multi-trajets (un véhicule peut effectuer plusieurs tournées dans la journée en revenant se recharger au dépôt).

2.3.1 Vue d’ensemble du modèle

Le modèle présenté ci-après est un *Problème de Tournées de Véhicules Multi-Trajets avec Fenêtres de Temps et Capacités Hétérogènes* (MT-VRPTW), une extension du problème de tournées de véhicules classique [2, 19]. Nous présentons les données, les variables de décision, l’objectif et les contraintes du modèle, puis une description du flux de résolution, incluant la décomposition multi-trajets avec temps de rechargement, la compatibilité de type de véhicule (affectation des commandes sensibles à la température aux véhicules adaptés) et un mode optionnel de fenêtres de temps souples. Le modèle est résolu par une métaheuristique (Recherche Locale Guidée [20, 21] appliquée à une solution initiale construite par l’heuristique du plus proche arc), motivée par la NP-difficulté du problème sous-jacent [3, 11].

Énoncé du Problème

Un dépôt unique doit livrer un ensemble de commandes clients dans une journée de travail, à l’aide d’une flotte limitée et contrainte en capacité. Chaque commande possède une localisation géographique, une demande en poids et en volume, et une fenêtre de temps durant laquelle elle peut être servie. Chaque véhicule a une capacité en poids et une capacité en volume, et doit commencer et terminer chaque trajet au dépôt dans les heures d’ouverture de celui-ci. Un véhicule peut effectuer plus d’un trajet par jour : après être revenu au dépôt, il peut être rechargé (ce qui prend un temps de chargement fixe) puis renvoyé en tournée.

L’objectif est de servir le plus de commandes possible tout en minimisant, par priorité lexicographique, d’abord le nombre de véhicules utilisés puis la distance totale parcourue. Les commandes qui ne peuvent pas être servies de façon réalisable peuvent rester non servies, moyennant une pénalité. Cette structure relève de la famille des problèmes de tournées de véhicules avec fenêtres de temps (VRPTW) [18, 19].

Notations et Données

2.3.2 Ensembles et indices

- $C = \{1, 2, \dots, n\}$ — l'ensemble des commandes clients.
- Le nœud 0 — le dépôt. L'ensemble des nœuds est $N = \{0\} \cup C$.
- $A = \{(i, j) : i, j \in N, i \neq j\}$ — l'ensemble des arcs orientés.
- $K = \{1, 2, \dots, m\}$ — l'ensemble des véhicules disponibles (après limitation de la flotte au nombre de chauffeurs disponibles ; voir la Section 2.3.11).
- T — l'ensemble des types de véhicule (régimes de température), p. ex. {normal, réfrigéré, congelé, etc.}

2.3.3 Paramètres

- $d_{ij} \in \mathbb{R}_+$ — distance routière (km) du nœud i au nœud j , obtenue du service de routage (OSRM en production).
- $t_{ij} \in \mathbb{Z}_{\geq 0}$ — temps de trajet routier (secondes) du nœud i au nœud j .
- $s_i \in \mathbb{Z}_{\geq 0}$ — temps de service (manutention) au nœud i . À chaque client $s_i = \sigma$ (une constante, p. ex. $\sigma = 600$ s = 10 min). Au dépôt $s_0 = \delta$, le *temps de service au dépôt* (chargement/rechargement), une constante configurable ; voir la Remarque 2.
- $q_i^w, q_i^v \in \mathbb{R}_+$ — demande en poids et en volume de la commande i ($q_0^w = q_0^v = 0$).
- $Q_k^w, Q_k^v \in \mathbb{R}_+$ — capacité en poids et en volume du véhicule k .
- $\tau_k \in T$ — type (régime de température) du véhicule k .
- $\hat{\tau}_i \in T \cup \{\emptyset\}$ — type requis par la commande i . La valeur $\emptyset$ (aucune exigence explicite) est traitée comme « normal » ; voir la Remarque 6.
- $[e_i, \ell_i]$ — la fenêtre de temps de la commande i , en secondes depuis minuit, où e_i est l'heure de début de service au plus tôt et ℓ_i au plus tard.
- $[E, L]$ — la fenêtre d'ouverture du dépôt (E = ouverture, L = fermeture), en secondes depuis minuit.
- $\delta \in \mathbb{Z}_{\geq 0}$ — le temps de service au dépôt (chargement en début de trajet ; de façon équivalente, le temps de rechargement/rotation entre deux trajets consécutifs d'un même véhicule, ces deux opérations étant la même opération physique au quai). Configurable ; valeur par défaut $\delta = 3600$ s.
- $\alpha \in \mathbb{R}_+$ — coût par véhicule utilisé.
- $\beta \in \mathbb{R}_+$ — coût par kilomètre parcouru.
- $P \in \mathbb{R}_+$ — pénalité pour une commande laissée non servie.

Remarque 1 (Prétraitement des fenêtres de temps). *Une fenêtre client ouvrant avant le dépôt, $e_i < E$, est ramenée à $e_i \leftarrow E$, puisqu’aucun véhicule ne peut partir avant l’ouverture du dépôt. Si, après ce recalage, $\ell_i \leq e_i$, la fenêtre est élargie à celle du dépôt afin d’éviter un intervalle vide (infaisable); une telle commande ne sera servie que si la temporisation de la tournée le permet, ou sera sinon laissée non servie.*

Remarque 2 (Temps de service au dépôt et plancher de départ unifié). *Charger un véhicule en début de trajet et le recharger entre deux trajets sont la même opération physique au quai; le modèle utilise donc un unique paramètre configurable δ pour les deux. Un véhicule ne peut pas partir avant d’avoir été chargé, et le chargement ne peut commencer avant que le véhicule ne soit présent au dépôt et que le dépôt ne soit ouvert. Cela donne une règle unique régissant l’heure de départ $a_0^{(k)}$ de chaque trajet du véhicule k :*

$$a_0^{(k)} \geq b_k + \delta, \quad (2.1)$$

où b_k est l’instant à partir duquel le chargement de ce trajet peut commencer :

$$b_k = \begin{cases} E, & \text{premier trajet (chargement possible dès l’ouverture),} \\ \text{heure de retour de } k, & \text{trajets suivants (chargement dès le retour de } k). \end{cases}$$

Ainsi le **premier trajet** ne peut partir avant $E + \delta$ (le dépôt doit être ouvert assez longtemps pour charger), et un **second trajet** ne peut partir avant son heure de disponibilité $r_k = (\text{heure de retour de } k) + \delta$, ce qui est exactement (2.1) avec b_k fixé à l’heure de retour.

Comptabilisé une seule fois. Comme le flux multi-trajets (Section 2.3.11) résout chaque trajet comme une instance de modèle distincte, δ est réalisé comme le **plancher de départ** (2.1) de chaque trajet, et le temps de service au dépôt à l’intérieur d’une tournée est pris égal à 0 ($s_0 = 0$ dans la propagation), de sorte que le chargement est comptabilisé par le plancher et jamais une seconde fois dans la tournée. En pratique, le plancher du premier trajet est réalisé en décalant l’heure d’ouverture vue par le solveur de E à $E + \delta$.

Chargement au plus tard (présentation uniquement). La contrainte (2.1) ne fixe que le départ au plus tôt; elle n’oblige pas le véhicule à être chargé dès b_k puis à attendre. Si le départ choisi est $d \geq b_k + \delta$, le chargement peut être rapporté comme occupant l’intervalle $[d - \delta, d]$ — c’est-à-dire un chargement au plus tard, terminé juste avant le départ. Il s’agit d’un choix de présentation à l’étape d’extraction de la solution; il ne change pas l’ensemble réalisable.

Variables de Décision

- $x_{ijk} \in \{0, 1\}$ — vaut 1 si le véhicule k se déplace directement du nœud i au nœud j , et 0 sinon.
- $y_k \in \{0, 1\}$ — vaut 1 si le véhicule k est utilisé (effectue au moins un trajet), et 0 sinon.
- $z_i \in \{0, 1\}$ — vaut 1 si la commande i est *servie*, et 0 si elle est laissée non servie.
- $a_i \in \mathbb{Z}_{\geq 0}$ — l'instant où le service *commence* au nœud i (l'heure d'arrivée, éventuellement après attente).
- $w_i^w, w_i^v \in \mathbb{R}_{\geq 0}$ — la charge cumulée (poids, volume) à l'arrivée au nœud i .

Fonction Objectif

L'optimisation minimise un coût pondéré unique combinant la taille de la flotte, la distance et les pénalités de commandes non servies :

$$\min \underbrace{\alpha \sum_{k \in K} y_k}_{\text{coût de flotte}} + \underbrace{\beta \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} x_{ijk}}_{\text{coût de distance}} + \underbrace{P \sum_{i \in C} (1 - z_i)}_{\text{pénalité de non-service}} . \quad (2.2)$$

Les deux premiers termes reproduisent l'objectif multicritère classique du VRP $\min(\alpha \sum_k y_k + \beta \sum_{ijk} d_{ij} x_{ijk})$ [19]. Le troisième terme, actif uniquement lorsque le service partiel est autorisé, valorise la décision d'abandonner une commande ; ce mécanisme correspond aux disjonctions du solveur [9].

Calibrage des coûts. Les magnitudes relatives de α , β et P encodent l'ordre de priorité voulu et doivent satisfaire deux conditions :

$$\alpha \gg \beta \cdot (\text{diamètre du réseau en km}), \quad (2.3)$$

$$P > \alpha. \quad (2.4)$$

La condition (2.3) fait que l'activation d'un véhicule supplémentaire domine toute économie de distance, de sorte que le modèle minimise d'abord le nombre de véhicules puis la distance (comportement lexicographique par pondérations scalaires). La condition (2.4) assure que la pénalité d'abandon dépasse le coût fixe d'activation d'un véhicule ; sans elle, le solveur pourrait abandonner tout un camion de commandes plutôt que d'activer ce véhicule, car le coût fixe économisé α dépasserait les pénalités accumulées. Dans l'implémentation, on utilise $P = \alpha + 1000\beta$ (en unités de coût mises à l'échelle).

Calibrage strict (tenant compte de la distance). La condition (2.4) ne tient pas compte de la distance parcourue. À elle seule, elle n'oblige pas à servir une commande

unique très éloignée : si l'atteindre coûte $\alpha + \beta d_{0i} > P$, le solveur l'abandonnera rationnellement, traitant P comme le *seuil de valeur* en deçà duquel une commande ne vaut pas un trajet dédié. Pour servir toute commande *réalisable* quelle que soit la distance, on renforce (2.4) en

$$P > \alpha + \beta D, \quad (2.5)$$

où D borne la plus longue tournée possible d'un véhicule. La valeur par défaut $P = \alpha + 1000\beta$ réalise (2.5) sous l'hypothèse qu'aucune tournée ne dépasse ≈ 1000 km, ce qui convient à une exploitation à l'échelle urbaine ou régionale.

Remarque 3 (Mise à l'échelle entière). *Le solveur exige des coûts d'arc entiers [9]. Tous les coûts réels sont convertis en entiers via un unique facteur d'échelle S (unités de coût par km) : une distance d_{ij} devient $\lfloor d_{ij} S \rfloor$, le coût fixe de véhicule devient $\lfloor \alpha S \rfloor$, et la pénalité devient $\lfloor P S \rfloor$. Utiliser un unique facteur pour tous les termes de coût est essentiel ; mélanger les échelles fausse silencieusement les arbitrages de (2.2).*

Contraintes

La formulation suit les formulations classiques en flot du VRP [18, 19].

2.3.4 Contraintes de routage et de degré

Chaque client servi est entré et quitté exactement une fois ; un client non servi n'est pas visité :

$$\sum_{k \in K} \sum_{j \in N, j \neq i} x_{ijk} = z_i, \quad \forall i \in C, \quad (2.6)$$

$$\sum_{k \in K} \sum_{j \in N, j \neq i} x_{jik} = z_i, \quad \forall i \in C. \quad (2.7)$$

La conservation du flot assure que tout ce qui entre dans un nœud en ressort sur le même véhicule :

$$\sum_{j \in N, j \neq h} x_{jhk} = \sum_{j \in N, j \neq h} x_{hjk}, \quad \forall h \in C, \forall k \in K. \quad (2.8)$$

2.3.5 Utilisation des véhicules et degré au dépôt

Un véhicule utilisé quitte le dépôt et y revient ; un véhicule inutilisé reste à l'arrêt :

$$\sum_{j \in C} x_{0jk} = y_k, \quad \forall k \in K, \quad (2.9)$$

$$\sum_{i \in C} x_{i0k} = y_k, \quad \forall k \in K. \quad (2.10)$$

(Dans le flux multi-trajets de la Section 2.3.11, chaque trajet est un problème de routage distinct ; (2.9)–(2.10) s’appliquent donc par trajet.)

2.3.6 Contraintes de capacité

La charge cumulée est propagée le long de chaque arc et bornée par la capacité du véhicule qui le parcourt. Pour tout $(i, j) \in A$ et $k \in K$:

$$x_{ijk} = 1 \implies w_j^w = w_i^w + q_j^w, \quad w_i^w \leq Q_k^w, \quad (2.11)$$

$$x_{ijk} = 1 \implies w_j^v = w_i^v + q_j^v, \quad w_i^v \leq Q_k^v. \quad (2.12)$$

Le poids et le volume sont suivis comme deux dimensions de capacité indépendantes ; une affectation n’est réalisable que si elle respecte les *deux*.

2.3.7 Propagation temporelle et journée de travail

Les heures de début de service sont propagées le long des arcs, en ajoutant le temps de trajet et de service. Pour tout $(i, j) \in A$, $k \in K$:

$$x_{ijk} = 1 \implies a_j \geq a_i + s_i + t_{ij}. \quad (2.13)$$

L’inégalité (plutôt que l’égalité) autorise l’attente : un véhicule arrivant avant e_j peut attendre l’ouverture de la fenêtre. Toute l’activité est confinée à la journée de travail du dépôt :

$$E \leq a_i \leq L, \quad \forall i \in N. \quad (2.14)$$

L’heure de *départ* du dépôt est bornée plus finement que le générique $a_i \geq E$: un véhicule ne peut partir avant d’être chargé, donc son départ de premier trajet vérifie $a_0^{(k)} \geq E + \delta$, le plancher de la Remarque 2, équation (2.1).

Remarque 4 (Élimination implicite des sous-tours). *Les contraintes de flot (2.6)–(2.8) n’interdisent pas, à elles seules, les sous-tours déconnectés. C’est la propagation temporelle (2.13) qui joue ce rôle : comme l’heure de service croît strictement le long de chaque arc ($a_j \geq a_i + s_i + t_{ij}$ avec $s_i + t_{ij} > 0$), aucun nœud ne peut se précéder lui-même dans le temps, ce qui rend tout cycle fermé ne passant pas par le dépôt infaisable. La variable temporelle a_i joue donc le rôle des variables d’ordre de Miller–Tucker–Zemlin [13] : elle élimine les sous-tours sans qu’il soit nécessaire d’ajouter des contraintes dédiées.*

Remarque 5 (Horizon en temps absolu). *La variable a_i est un temps absolu d’horloge (secondes depuis minuit), et non une durée écoulée. Par conséquent, la borne supérieure qui plafonne la dimension temporelle doit être l’heure de fermeture du dépôt L , et non la durée de journée $L - E$. Cette distinction est cruciale pour les trajets démarrant tard dans*

la journée (Section 2.3.11) : borner par $L - E$ tronquerait silencieusement toute tournée dont le temps absolu dépasse $E + (L - E)$ calculé à partir d'un départ matinal, rendant les départs tardifs faussement infaisables.

2.3.8 Fenêtres de temps — mode dur (par défaut)

Dans la formulation par défaut, les fenêtres sont dures : le service ne peut commencer ni avant e_i ni après ℓ_i [18].

$$z_i = 1 \implies e_i \leq a_i \leq \ell_i, \quad \forall i \in C. \quad (2.15)$$

Une commande dont la fenêtre ne peut être respectée est forcée à $z_i = 0$ (non servie) et encourt la pénalité P dans (2.2).

2.3.9 Fenêtres de temps — mode souple (optionnel)

Les fenêtres dures sont pessimistes pour la logistique réelle, où de nombreuses fenêtres expriment une *préférence* plutôt qu'une règle stricte. Dans le mode souple optionnel, une commande peut être servie après ℓ_i moyennant une pénalité de retard, jusqu'à une limite de grâce $\ell_i + g$ (g = période de grâce). L'heure au plus tôt e_i reste dure (un véhicule ne peut servir avant l'ouverture du client). On remplace la borne supérieure de (2.15) par une borne souple pénalisée et on augmente l'objectif. Soit le retard de la commande i :

$$\lambda_i = \max(0, a_i - \ell_i), \quad 0 \leq \lambda_i \leq g. \quad (2.16)$$

Le service est permis sur $[e_i, \ell_i + g]$, et l'objectif gagne un terme de retard :

$$\min \quad (\text{Éq. 2.2}) + \gamma \sum_{i \in C} \lambda_i z_i, \quad (2.17)$$

où $\gamma > 0$ est le coût de retard par unité de temps. Au-delà de $\ell_i + g$, la commande ne peut qu'être laissée non servie (ou reportée à un trajet ultérieur). Cette borne souple est réalisée dans le solveur par une borne supérieure pénalisée sur la dimension temporelle [9].

Calibrage de la pénalité de retard. La pente de retard γ est fixée en exigeant qu'un retard égal à *toute* la période de grâce g coûte approximativement autant que l'abandon de la commande :

$$\gamma \cdot g \approx P \implies \gamma = \frac{P}{g}. \quad (2.18)$$

Cela produit l'ordre de préférence voulu

à l'heure $\prec$ légèrement en retard $\prec$ très en retard $\prec$ abandon.

Le choix entre borne dure (2.15) et borne souple (2.16)–(2.17) est une propriété par commande ; une même instance peut donc mélanger clients stricts et flexibles.

2.3.10 Contrainte de type de véhicule (régime de température)

Chaque véhicule possède un *unique* type $\tau_k \in T$ définissant le régime de température de *tout* son chargement sur une tournée (ambiant, réfrigéré, surgelé). Comme toutes les commandes d’une même tournée partagent le même compartiment à la même température, elles doivent toutes exiger ce même régime. La contrainte est une égalité de type : la commande i ne peut être servie que par un véhicule k tel que $\tau_k = \hat{\tau}_i$:

$$x_{ijk} = 1 \implies \tau_k = \hat{\tau}_i, \quad \forall i \in C, \forall j \in N, \forall k \in K. \quad (2.19)$$

En pratique, pour chaque commande, on restreint l’ensemble des véhicules admissibles à son nœud à ceux du type requis [9].

Remarque 6 (Type unique plutôt que compétences multiples). *Le modèle attribue à chaque véhicule un seul type, et non un ensemble de compétences. Ce choix reflète le fait que la température est une propriété du compartiment entier : on ne peut pas mélanger des marchandises ambiantes et réfrigérées dans le même camion. Une commande sans type explicite ($\hat{\tau}_i = \emptyset$) est traitée comme « normal » (ambiant) ; elle ne peut donc pas voyager dans un camion réfrigéré ou congélateur qui roule en froid pour ses autres commandes. Ce modèle strict évite qu’une marchandise ambiante soit refroidie, ou qu’une marchandise froide soit réchauffée. Son coût est une réduction de la flexibilité : si la flotte ne comporte aucun véhicule du type requis, la commande est laissée non servie plutôt que mal affectée.*

2.3.11 Domaines des variables

$$x_{ijk}, y_k, z_i \in \{0, 1\}; \quad a_i \in \mathbb{Z}_{\geq 0}; \quad w_i^w, w_i^v \in \mathbb{R}_{\geq 0}. \quad (2.20)$$

Remarque 7 (Véhicule inutilisé et commandes non servies peuvent coexister, mais seulement sous infaisabilité). *Une solution peut présenter un véhicule inutilisé ($y_k = 0$) en même temps que des commandes non servies ($z_i = 0$). Sous le calibrage strict tenant compte de la distance (condition (2.5)), ce n’est pas un signe de paresse mais d’infaisabilité réelle. Deux cas se présentent :*

1. **Commandes servables, véhicule inutilisé — exclu.** *Si un véhicule inutilisé pouvait servir une commande non servie (capacité, fenêtre et type le permettant), alors l’activer coûte $\alpha + \beta d$ tandis que l’abandonner coûte P ; sous (2.5), servir reste moins coûteux que l’abandon. Le solveur active donc le véhicule. Cette combinaison ne peut donc survenir à l’optimum.*

2. **Commandes infaisables, véhicule inutilisé — attendu.** Si aucun véhicule disponible ne peut servir les commandes — demande cumulée dépassant toute capacité restante (2.11)–(2.12), fenêtres déjà fermées (2.15), type incompatible (2.19), ou hors de portée dans la journée (2.14) — alors activer un véhicule n’aide pas, et le seul choix réalisable est $z_i = 0$.

Par conséquent, un véhicule inutilisé aux côtés de commandes non servies est un signal de sortie significatif : la flotte restante ne peut servir ces commandes sous les contraintes actuelles. C’est un rapport correct, non un défaut de modélisation.

Complexité et Choix de la Méthode

Le modèle généralise le problème de tournées de véhicules avec fenêtres de temps, lui-même une généralisation du problème du voyageur de commerce, et est donc **NP-difficile** [3, 11]. Les méthodes exactes (séparation et évaluation, coupes, génération de colonnes [19]) ne résolvent que des instances modestes dans des temps pratiques ; pour une instance d’environ 67 clients sous une limite de temps stricte, un optimum exact n’est pas atteignable de façon fiable.

On résout donc le modèle par une **métaheuristique** [4] : une heuristique de construction (plus proche arc) produit une solution initiale réalisable, améliorée ensuite par la **Recherche Locale Guidée** (GLS) [20, 21]. La GLS explore des mouvements de voisinage (relocalisation, échange, 2-opt, Or-opt) et échappe aux optima locaux en pénalisant les caractéristiques (arcs) récurrentes dans les minima locaux, le tout sous une limite de temps. On échange ainsi la garantie d’optimalité contre des solutions de haute qualité dans un temps borné et prévisible — le choix d’ingénierie approprié pour un TMS interactif. Cette approche est celle recommandée pour le routage dans OR-Tools [9].

Remarque 8 (Lien avec l’ILP exact). La formulation (2.2)–(2.15) est un véritable programme linéaire en nombres entiers et pourrait, pour de petites instances, être soumise à un solveur exact (p. ex. CPLEX [10]), ce qui sert à valider la métaheuristique sur des cas où l’optimum est calculable. Les contraintes d’implication (2.11)–(2.13) sont linéarisées de manière standard (grand-M ou propagation à la Miller–Tucker–Zemlin [13]) lorsqu’un ILP explicite est requis.

Flux de Résolution

Le système résout la journée en deux phases. Chaque phase est une instance du modèle mono-trajet ci-dessus ; le comportement multi-trajets émerge de l’enchaînement des phases avec temps de rechargement.

Étape 0 — Limitation et sélection de la flotte

Le nombre de véhicules est plafonné au nombre de chauffeurs disponibles (un chauffeur par véhicule). Lorsque les véhicules sont plus nombreux que les chauffeurs, on procède à une sélection intelligente basée sur les types requis par les commandes : on compte d'abord le nombre de commandes par type (NORMAL, REFRIGERE, CONGELATEUR), puis on sélectionne en priorité les véhicules correspondant aux types les plus demandés, en privilégiant les véhicules de plus grande capacité combinée pour chaque type.

Étape 1 — Construction des matrices

Pour les $|N|$ emplacements (dépôt en premier), le service de routage (OSRM) renvoie la matrice des distances $[d_{ij}]$ (km) et la matrice des temps $[t_{ij}]$ (s). Les valeurs réelles sont converties en unités de coût entières via l'unique facteur d'échelle S .

Étape 2 — Assemblage du modèle (Phase 1)

On construit le modèle de routage sur les n clients et m véhicules : on enregistre le rappel de distance (coût d'arc = $\lfloor \beta d_{ij} S \rfloor$) ; on fixe le coût fixe de véhicule $\lfloor \alpha S \rfloor$; on ajoute la dimension temporelle avec rappel $t_{ij} + s_i$, du mou pour l'attente et un horizon L ; on impose les bornes de fenêtre (2.15) (ou les bornes souples (2.16)–(2.17) le cas échéant) ; on ajoute les deux dimensions de capacité (2.11)–(2.12) ; pour chaque commande exigeant un type, on restreint la variable véhicule de son nœud au sous-ensemble compatible (2.19) ; et, si le service partiel est autorisé, on ajoute une disjonction par client de pénalité $\lfloor P S \rfloor$ (réalisant la variable z_i). L'heure de départ de chaque véhicule est plancherée par la borne $a_0^{(k)} \geq E + \delta$ de (2.1), et majorée par L .

Étape 3 — Recherche (Phase 1)

On génère la solution initiale par l'heuristique du plus proche arc, puis on l'améliore par Recherche Locale Guidée [21] sous la limite de temps (ajustée à la taille de l'instance).

Étape 4 — Extraction (Phase 1)

Pour chaque véhicule utilisé, on parcourt sa tournée depuis le dépôt, en enregistrant les arrêts ordonnés, les heures d'arrivée a_i , la distance et la charge par tournée, et l'heure de retour au dépôt. Le départ rapporté est le départ *au plus tard* (cf. Remarque 2), plancheré à $E + \delta$. Les clients atteints par aucun véhicule sont enregistrés comme non servis.

Étape 5 — Rechargement et disponibilité pour un second trajet

S'il reste des commandes non servies, on calcule pour chaque véhicule k son *heure de disponibilité*

$$r_k = (\text{heure de retour de } k \text{ en Phase 1}) + \delta, \quad (2.21)$$

soit le plancher unifié (2.1) avec b_k fixé à l'heure de retour. Un véhicule est éligible à un second trajet uniquement s'il est disponible avec assez de journée restante pour être utile, c.-à-d. $r_k < L - \theta$ pour une marge θ (p.ex. 1 h), et s'il n'est pas trop en retard sur le groupe : on exclut tout véhicule dont r_k dépasse de plus d'une heure la médiane des heures de disponibilité de la flotte (un véhicule désynchronisé du groupe ne coordonne pas utilement une seconde vague).

Étape 6 — Filtre d'atteignabilité (candidats Phase 2)

Soit $r_{\min} = \min_{k \text{ éligible}} r_k$. Une commande non servie i n'est candidate à un second trajet que si un véhicule éligible peut réalistement la servir : en partant à $r_{\min}$, l'aller-retour doit tenir dans la fenêtre et dans la journée,

$$r_{\min} + t_{0i} \leq \ell_i \quad (\text{ou } \ell_i + g \text{ en mode souple}), \quad (2.22)$$

$$r_{\min} + t_{0i} + s_i + t_{i0} \leq L. \quad (2.23)$$

Ce filtre garde l'ensemble candidat cohérent avec la temporisation qu'imposera le solveur de Phase 2.

Étape 7 — Assemblage et recherche (Phase 2)

On analyse les types requis par les commandes candidates non servies et on sélectionne les véhicules appropriés parmi *tous* les véhicules disponibles (pas seulement ceux utilisés en Phase 1), permettant aux chauffeurs de changer de véhicule si les types requis diffèrent de la Phase 1. On construit un nouveau modèle sur les commandes candidates et les véhicules sélectionnés, identique à la Phase 1, sauf que chaque véhicule k (conduit par un chauffeur assigné) reçoit un départ au plus tôt r_k (2.21) basé sur l'heure de disponibilité du chauffeur. Cela garde le plan exécutable : aucun chauffeur ne repart avant d'être revenu au dépôt, d'avoir rechargé, et d'avoir éventuellement changé de véhicule. On résout par la même métaheuristique sous une limite de temps plus courte.

Étape 8 — Combinaison et comptabilité

On concatène les tournées des Phases 1 et 2. La distance totale est la somme des deux phases. Une commande est rapportée non servie si et seulement si elle n'apparaît dans

aucune tournée des deux phases. Le même chauffeur physique effectue les deux trajets, même s'il utilise des véhicules différents entre Phase 1 et Phase 2 (retour au dépôt pour changement de véhicule si nécessaire).

Remarque 9 (Multi-trajets glouton vs. conjoint). *Le schéma en deux phases est une heuristique multi-trajets séquentielle (gloutonne) : la Phase 1 fixe ses tournées sans anticiper la Phase 2. Un modèle multi-trajets pleinement conjoint — où une seule optimisation permet à chaque véhicule de revenir au dépôt, recharger, puis continuer — pourrait trouver de meilleurs plans combinés, au prix d'un modèle bien plus grand et difficile (nœuds dépôt répliqués, contraintes de liaison de trajets, rechargement modélisé dans la dimension temporelle). Le schéma séquentiel est choisi comme compromis pragmatique entre qualité, temps de calcul et complexité d'implémentation ; le modèle conjoint est identifié comme travail futur.*

De plus, le schéma est borné à deux trajets (un premier puis un second). En logistique urbaine dense, un véhicule léger peut effectuer trois trajets ou plus dans une journée. Le flux se généralise naturellement en une boucle : tant qu'il reste des commandes non servies atteignables et des véhicules disponibles avec assez de journée restante, on lance un trajet supplémentaire, chaque vague utilisant le plancher de disponibilité r_k (2.21) de la vague précédente. Pour l'exploitation visée (camions, contraintes de journée de travail et de rechargement δ), deux trajets épuisent en pratique la plage horaire utile ; la généralisation à N trajets est donc identifiée comme extension, sans impact pratique attendu à cette échelle.

Hypothèses et Limitations

Le modèle assume des livraisons **intra-urbaines ou proches de la ville**, c.-à-d. que chaque client est atteignable, servi et le retour effectué dans une seule journée de travail du dépôt. Les livraisons longue distance ou hors ville, dont le temps d'aller-retour approche ou dépasse la journée de travail, sont hors champ : une telle commande serait soit laissée non servie, soit monopoliserait un véhicule pour la journée, et le mécanisme multi-trajets (qui suppose un retour au dépôt pour recharger) ne s'y applique pas. Modéliser les livraisons longue distance comme des tournées dédiées, ou prendre en charge une planification multi-jours avec fenêtres de temps assouplies, est identifié comme travail futur. D'autres extensions hors champ incluent les temps de trajet dépendant de l'heure (heures de pointe), les réglementations sur les temps de conduite des chauffeurs, et l'appariement collecte–livraison.

Récapitulatif des Paramètres

Symbole	Signification	Valeur par défaut
α	coût par véhicule utilisé	10000
β	coût par km	1
P	pénalité de non-service	$\alpha + 1000\beta$
S	échelle de coût (unités/km)	100
σ	temps de service par client	600 s (10 min)
δ	temps de service au dépôt (charge / recharge)	configurable (3600 s)
τ_k	type du véhicule k	dans T (défaut normal)
$\hat{\tau}_i$	type requis par la commande i	$T \cup \{\emptyset\}$ ($\emptyset$ = normal)
g	période de grâce (souple)	7200 s (2 h)
γ	coût de retard par seconde	P/g
θ	marge de fin de journée (Étape 5)	3600 s (1 h)
$[E, L]$	fenêtre d'ouverture du dépôt	configuration

TABLE 2.1 – Paramètres du modèle et leurs valeurs par défaut d'implémentation.

2.3.12 Correspondance entre modèle et implémentation

Le tableau 2.2 établit la correspondance entre les composants mathématiques du modèle VRPTW formel et leur traduction en appels à la bibliothèque OR-Tools. Cette correspondance montre comment le code reprend les contraintes principales du modèle théorique.

Composant Mathématique	Implémentation OR-Tools
Variables x_{ijk} (affectation arcs)	Implicite dans <code>RoutingModel</code> (graphe interne)
Distance d_{ij} (matrice)	Callback <code>distance_callback</code> avec <code>scaling × 100</code>
Capacité poids $\sum q_i \cdot x_{ijk} \leq Q_k$	<code>AddDimensionWithVehicleCapacity</code> (dimension "Capacity_Weight")
Capacité volume $\sum v_i \cdot x_{ijk} \leq V_k$	<code>AddDimensionWithVehicleCapacity</code> (dimension "Capacity_Volume")
Fenêtres temps $a_i \leq T_{ik} \leq b_i$	<code>CumulVar().SetRange(min, max)</code> sur dimension "Time"
Objectif distance $\sum d_{ij} \cdot x_{ijk}$	<code>SetArcCostEvaluatorOfAllVehicles</code>
Objectif nb véhicules $\alpha \cdot \sum y_k$	<code>SetFixedCostOfAllVehicles</code>

TABLE 2.2 – Mapping entre modèle mathématique et API OR-Tools

Note sur le scaling : OR-Tools exige des valeurs entières pour les coûts. Les distances (en km) sont donc multipliées par 100 avant d'être passées au solveur, ce qui donne une granularité de 0,01 km tout en évitant les erreurs de troncature.

2.4 Conclusion

Ce chapitre a présenté la conception globale du système selon deux perspectives complémentaires.

La **conception logicielle** (section 2.1) a formalisé l’architecture via des diagrammes UML. Le diagramme de cas d’utilisation a identifié les interactions entre les quatre acteurs (Expéditeur, Dispatcheur, Chauffeur, Administrateur) et le système, en détaillant les 20+ fonctionnalités principales. Le diagramme de classes a structuré le modèle de données en 8 entités principales (Utilisateur, Warehouse, Vehicule, Commande, Tournee, StopTournee, Anomalie, TacheOptimisation) avec leurs attributs, relations, et contraintes d’intégrité. Les diagrammes de séquence ont illustré trois flux critiques (création de commande, optimisation asynchrone, confirmation de livraison), en mettant en évidence les points clés : géolocalisation automatique, découplage asynchrone via Celery/Redis, et optimisation réseau par batching des requêtes.

La **conception mathématique** (section 2.2) a formalisé le problème d’optimisation comme un VRPTW multi-contraint (capacités doubles, types de véhicules, fenêtres de temps souples et dures, multi-trajets). Le modèle MILP définit les paramètres, variables de décision, fonction objectif, et contraintes. La logique multi-trajets permet d’augmenter le taux de service en autorisant plusieurs tournées par véhicule dans la même journée. Le choix d’une métaheuristique (Recherche Locale Guidée via OR-Tools) est motivé par la NP-difficulté du problème et les contraintes de temps de calcul : limite OR-Tools par phase de 30 à 120s selon la taille, pour un temps d’exécution total mesuré de 30s à 4.5min selon les instances.

Ces deux conceptions convergent vers une implémentation unique : les classes logicielles représentent les principales entités métier, et le code du solveur encode les contraintes structurantes du modèle formel. Cette cohérence permet au système informatique de traiter le problème métier identifié au chapitre 1, tout en restant soumis aux limites d’une métaheuristique sans garantie d’optimalité globale.

Le chapitre suivant détaillera la réalisation technique : choix technologiques (Python, OR-Tools, FastAPI, React, PostgreSQL, OSRM), évaluation des services de routage (Google vs OSRM), implémentation du solveur, interfaces utilisateur, déploiement en production, et évaluation des performances (temps de calcul, consommation CPU/RAM, qualité des solutions).

Chapitre 3

Réalisation et Évaluation

3.1 Introduction

Ce chapitre présente la réalisation concrète du système TMS, en détaillant les choix technologiques, l'architecture logicielle, l'implémentation du solveur d'optimisation, et l'évaluation des performances. Nous montrons comment les conceptions logicielle et mathématique du chapitre 2 se traduisent en un système opérationnel déployé en production.

La section 3.1 décrit la pile technologique complète (backend, frontend, base de données, services externes). La section 3.2 détaille l'implémentation du moteur d'optimisation et les optimisations réseau critiques. La section 3.3 présente les interfaces utilisateur pour chaque acteur. Enfin, la section 3.4 évalue les performances du système (temps de calcul, consommation mémoire, qualité des solutions).

3.2 Architecture Technique et Choix Technologiques

3.2.1 Vue d'ensemble de l'architecture

Le système adopte une architecture 3-tiers, séparant présentation, logique métier, et persistance des données.

Couche Présentation : Application web monopage (SPA) développée en React 19, servie par Nginx en production. L'interface communique avec le backend via API REST.

Couche Métier : API REST développée en FastAPI [17] (Python 3.12), exposant les endpoints CRUD et métier. Un worker Celery [1] dédié exécute les tâches d'optimisation de manière asynchrone.

Couche Données : Base de données relationnelle PostgreSQL [15] 16, garantissant la cohérence transactionnelle et la persistance. Redis sert de broker pour Celery et de cache pour les sessions.

Services Externes : Google Geocoding API (conversion adresse → coordonnées GPS, avec repli Nominatim), OSRM auto-hébergé (calcul de matrices de distances/temps pour

l'optimisation, affichage des itinéraires sur carte).

3.2.2 Stack Backend

FastAPI (Python 3.12). [17] FastAPI a été choisi pour ses performances élevées (comparable à Node.js grâce à l'asynchronisme), sa validation automatique des données via Pydantic, et sa génération automatique de documentation OpenAPI interactive (/docs). L'utilisation d'`async/await` permet de gérer efficacement les opérations I/O (requêtes base de données, appels API externes) sans bloquer le thread principal.

SQLAlchemy + Alembic. SQLAlchemy 2.0 en mode asynchrone (`AsyncSession`) gère l'ORM. Les modèles Python (Utilisateur, Commande, Tournee, etc.) sont alignés avec les tables PostgreSQL. Alembic gère les migrations de schéma de manière versionnée, facilitant l'évolution du modèle de données sans perte de données.

Celery + Redis. Celery orchestre l'exécution asynchrone des tâches d'optimisation. Redis sert de broker (file de messages) et de backend de résultats. Cette architecture découple l'API (qui répond immédiatement avec un `task_id`) et le calcul intensif (qui s'exécute en arrière-plan pendant 30s-4.5min selon la taille du problème). Le frontend interroge périodiquement le statut de la tâche via `GET /api/v1/optimisation/{id}/statut`.

Google OR-Tools. [9] Bibliothèque d'optimisation open-source de Google, utilisée pour résoudre le VRPTW via métaheuristique GLS. Elle permet de modéliser les contraintes de capacité, les fenêtres de temps et les affectations véhicule-client.

3.2.3 Stack Frontend

React 19 + TypeScript. [12] React permet de construire une interface réactive avec gestion d'état locale et globale. TypeScript apporte la sécurité des types, réduisant les erreurs à l'exécution et améliorant l'expérience développeur (autocomplétion, refactoring).

TanStack Query. Gère le cache, la synchronisation, et le polling des données API. Les requêtes sont automatiquement mises en cache et invalidées lors des mutations, réduisant les appels réseau redondants.

TailwindCSS. Framework CSS utilitaire permettant un développement rapide avec design cohérent. Les classes utilitaires (`flex`, `rounded-xl`, `bg-blue-500`) évitent l'écriture de CSS personnalisé.

Leaflet.js. Bibliothèque de cartographie interactive légère. Affiche les tournées sur une carte OpenStreetMap avec marqueurs colorés selon le statut des arrêts. OSRM auto-hébergé fournit les géométries d’itinéraire détaillées.

Recharts. Bibliothèque de graphiques React pour les tableaux de bord KPI (évolution OTD, utilisation de la flotte).

3.2.4 Base de Données et Persistance

PostgreSQL 16. SGBD relationnel robuste, supportant les transactions ACID, les contraintes d’intégrité référentielle, et les index performants. Déployé via Docker (`postgres:16-alpine`) pour reproductibilité. Les migrations Alembic garantissent la cohérence du schéma entre environnements (dev, test, prod).

Modèle de données. Le schéma relationnel compte 8 tables principales (cf. chapitre 2) avec clés étrangères, contraintes d’intégrité relationnelle, types énumérés PostgreSQL pour les statuts/rôles, contraintes `NOT NULL/UNIQUE` selon les champs, et index techniques sur les clés primaires ainsi que sur l’email utilisateur. Des index métier supplémentaires sur `date_livraison`, `statut` ou `warehouse_id` restent une optimisation possible si les volumes augmentent.

3.2.5 Services Externes

Géolocalisation : Google Geocoding API avec repli Nominatim. Le frontend utilise Google Geocoding API [8] en priorité pour convertir les adresses saisies par l’expéditeur en coordonnées GPS (lat, lon), avec restriction au pays Mauritanie (`country:MR`). Si Google ne retourne aucun résultat ou si la clé API est absente, Nominatim sert de repli avec filtrage pays (`countrycodes=mr`).

Routage : Évaluation et choix technique. Deux solutions de routage ont été évaluées pour le calcul des matrices de distances/temps nécessaires à l’optimisation :

- **Google Routes API v2 – Compute Route Matrix (phase de prototypage) :** [7]
 - *Avantages* : Données de trafic temps réel, service cloud établi, couverture mondiale.
 - *Inconvénients critiques* : Coût élevé ($\approx 14\$$ par optimisation de 100 commandes), rate limiting fréquent (erreurs HTTP 429), latence réseau importante ($\approx 150\text{ms}$ par requête), nécessité de chunking des requêtes (limite de 25 origines $\times$ 25 destinations par appel, soit jusqu’à 25 appels pour 100 commandes + dépôt), délais obligatoires entre requêtes (2s) pour éviter les blocages.

- *Impact mesuré* : Sur un test de 100 commandes, le temps total atteignait 1033s (17 min) dont $\approx 700s$ (68%) consacrés aux appels API et à la gestion des erreurs 429.
- **OSRM auto-hébergé (solution de production retenue)** : [16]
 - *Architecture* : Serveur OSRM conteneurisé (Docker) utilisant les données OpenStreetMap [14] de Mauritanie (extraits Geofabrik [5]) prétraitées avec l'algorithme MLD (Multi-Level Dijkstra).
 - *Table API* : Calcul de matrices complètes $N \times N$ de distances/temps via requête unique, sans chunking. Latence $< 1ms$, coût nul, pas de rate limiting.
 - *Route API* : Génération de polylines détaillées pour l'affichage des itinéraires sur carte (frontend Leaflet avec tuiles OpenStreetMap).
 - *Performance mesurée* : Même test de 100 commandes résolu en 251s (4.2 min), soit un gain de $4.1 \times$ par rapport à Google, grâce à l'absence d'interruptions.

Décision : OSRM a été retenu comme service de routage pour la production. Les données de trafic temps-réel de Google ne compensent pas les limitations opérationnelles mesurées (coût, latence, erreurs 429).

Évolution future possible : Pour les cas nécessitant des estimations d'arrivée très précises avec trafic temps-réel (ex : notification au client final), l'architecture peut être enrichie par des appels ponctuels à Google Directions API *après* l'optimisation, sur les K tournées finales uniquement (où $K \ll N$ commandes). Cette approche hybride réduirait les coûts de 99% ($14\$ \rightarrow 0.025\$$ par optimisation) tout en conservant le bénéfice du trafic temps-réel là où il est le plus utile : l'estimation finale communiquée au client.

3.3 Implémentation de l'Optimisation

3.3.1 Architecture du Moteur d'Optimisation

Le calcul d'optimisation s'exécute dans un processus Celery [1] dédié, découplé de l'API principale. Le flux complet :

1. Le dispatcher sélectionne commandes et véhicules via l'interface React [12].
2. `POST /api/v1/optimisation/lancer` crée une `TacheOptimisation` (statut `EN_ATTENTE`) et envoie le message Celery.
3. L'API retourne immédiatement `{tache_id: 42}` ($< 1s$).
4. Le worker Celery récupère la tâche depuis Redis, charge les données depuis PostgreSQL [15].
5. Le worker appelle OSRM (Table API) pour construire la matrice complète de distances/temps via requête unique.

6. Le worker initialise OR-Tools [9], configure contraintes et objectif, lance la résolution.
7. OR-Tools retourne une solution de bonne qualité (affectations véhicule-client, séquences).
8. Le worker crée les objets `Tournee` et `StopTournee` en base, met à jour les statuts des commandes.
9. Le frontend détecte `statut=TERMINEE` lors du prochain poll et affiche les résultats.

3.3.2 Optimisation Réseau : Réduction de Complexité

Problème initial. Pour n commandes, construire la matrice de distances complète nécessite $(n+1)^2$ paires origine-destination lorsque le dépôt est inclus. Avec 50 commandes : 51 nœuds, donc 2601 requêtes individuelles. À 100ms/requête : environ 260 secondes de latence réseau pure, rendant l’optimisation impraticable.

Solutions évaluées. Approche 1 : Batching avec Google Routes API v2 – Compute Route Matrix (phase de prototypage). Google Routes API v2 accepte des requêtes groupées : jusqu’à 625 paires (25 origines $\times$ 25 destinations) par appel. Pour 50 commandes (51 nœuds avec dépôt) :

- Matrice complète : $51 \times 51 = 2601$ paires
- Requêtes nécessaires : $\lceil 51/25 \rceil^2 = 9$ appels groupés
- Latence théorique : $9 \times 150\text{ms} = 1.35\text{s}$ (au lieu de plus de 4 minutes)
- Latence réelle mesurée : beaucoup plus élevée à cause des délais obligatoires (2s) entre requêtes pour éviter les erreurs 429, et de la gestion des erreurs de rate limiting

Approche 2 : Requête matricielle unique avec OSRM (solution retenue). OSRM Table API calcule la matrice $N \times N$ complète en une seule requête HTTP, sans limite ni chunking :

- Pour 100 commandes : 1 requête unique (matrice 101×101)
- Latence mesurée : $< 1\text{ms}$ (serveur local Docker)
- Aucun rate limiting, coût nul, pas de limite API externe observée dans nos tests

Résultat : L’approche OSRM élimine la complexité réseau, réduisant la latence API de plusieurs minutes à quelques millisecondes.

3.3.3 Implémentation du Solveur OR-Tools

Le solveur est implémenté dans `backend/app/solver/ortools_solver.py`. Étapes principales :

Étape 1 : Construction des matrices. Appel au service de routage (OSRM Table API) pour obtenir $[d_{ij}]$ (distances en km) et $[t_{ij}]$ (temps en secondes) via requête unique. Conversion en coûts entiers (OR-Tools exige des entiers) : multiplication par 100, ce qui donne une granularité de 0,01 km.

Étape 2 : Initialisation du modèle. Création du `RoutingIndexManager` (mapping nœuds $\leftrightarrow$ indices OR-Tools) et du `RoutingModel` (nombre de véhicules, nœuds de départ/retour).

Étape 3 : Contraintes de capacité. Deux dimensions (poids et volume) ajoutées via `AddDimensionWithVehicleCapacity`. Chaque commande consomme du poids ET du volume ; le véhicule ne peut dépasser sa capacité dans aucune dimension.

Étape 4 : Contraintes de temps. Dimension temporelle avec fenêtres de temps. Le solveur distingue fenêtres strictes (HARD, pénalité infinie si violation) et fenêtres souples (SOFT, pénalité proportionnelle au retard). En production actuelle, les commandes utilisent HARD par défaut car l'API ne permet pas encore de choisir le mode.

Étape 5 : Contraintes de types. Filtrage des affectations : une commande REFRI-GERE ne peut être servie que par un véhicule REFRIGERE ou CONGELATEUR.

Étape 6 : Multi-trajets. OR-Tools ne gère pas nativement les multi-trajets. Implémentation manuelle en deux phases : après une première résolution, les commandes non servies sont analysées par type (NORMAL, REFRIGERE, CONGELATEUR). La Phase 2 sélectionne les véhicules appropriés parmi *tous* les véhicules disponibles (pas seulement ceux utilisés en Phase 1) en fonction des types requis par les commandes restantes, permettant aux chauffeurs de changer de véhicule si nécessaire (p. ex. passer d'un véhicule NORMAL à un REFRIGERE). Les véhicules retournent au dépôt pour rechargement entre les phases.

Étape 7 : Fonction objectif. Minimiser la distance totale parcourue, avec pénalité secondaire pour le nombre de véhicules utilisés (favoriser la consolidation).

Étape 8 : Résolution. Métaheuristique GLS (Guided Local Search) avec limite de temps dynamique selon la taille du problème (30s pour < 20 commandes, 60s pour 20-50, 120s pour > 50). Stratégie de solution initiale : `PATH_CHEAPEST_ARC`. Le solveur retourne une solution réalisable de bonne qualité, sans garantie d'optimalité globale.

Étape 9 : Extraction de la solution. Parcours des routes retournées par OR-Tools, création des objets `Tournee` et `StopTournee` avec horodatages prévus, mise à jour des statuts (`AFFECTEE` ou `NON_AFFECTEE`).

3.3.4 Gestion des Erreurs et Cas Limites

- **Aucune solution trouvée :** Si OR-Tools échoue (contraintes trop strictes), la tâche passe en statut `ERREUR` avec message explicite dans `resultat_json`.
- **Timeout dépassé :** Les limites de temps sont dynamiques selon la taille du problème (30s pour petites instances, 60s pour moyennes, 120s pour grandes). Après expiration, OR-Tools retourne la meilleure solution trouvée.
- **Service de routage totalement indisponible :** Si OSRM échoue sur toutes les requêtes (conteneur arrêté, réseau coupé), l’optimisation est impossible et la tâche est marquée `ERREUR` avec message explicite. Le fallback `bulk→N×N` ne peut pas compenser une indisponibilité complète du service.

3.4 Interfaces Utilisateur

3.4.1 Vue d’ensemble

Le système propose 4 interfaces adaptées aux rôles utilisateur (cf. chapitre 1) :

- **Expéditeur :** Création et suivi de commandes.
- **Dispatcheur :** Orchestration (optimisation, gestion des tournées, KPI).
- **Chauffeur :** Exécution des tournées assignées.
- **Administrateur :** Gestion des utilisateurs, flotte, et entrepôts.

Le contrôle d’accès par rôle (RBAC) est appliqué côté backend (middleware FastAPI) et côté frontend (composant `ProtectedRoute`).

3.4.2 Interface Expéditeur

Création de commande. Formulaire avec validation en temps réel : adresse de livraison (géocodage automatique via Google Maps API avec repli Nominatim), poids, volume, date et fenêtre de temps, type de véhicule requis (optionnel). Le frontend bloque les valeurs manifestement invalides (poids/volume positifs, fenêtre horaire cohérente) avant soumission ; le backend applique les validations de type via les schémas Pydantic et vérifie notamment que la date de livraison est au moins celle du lendemain.

Nouvelle commande

Cr  er une nouvelle commande de transport

Entrep  t d'enl  vement

Entrep  t Principal Tevragh-Zeina - Nouakchott

Adresse de livraison

ETAK El Khair Soukoul 1, nouakchott

18.1277, -15.9635

Poids (kg)

100

Volume (m  )

150

Type de v  hicule

R  frig  r  

Laisser vide = NORMAL

Date de livraison

25/06/2026

Heure d'ouverture

08h

00

Heure de fermeture

22h

00

Cr  er la commande

Annuler

FIGURE 3.1 – Interface de cr  ation de commande (Exp  diteur)

Liste des commandes. Tableau pagin   avec filtres par statut et date. Chaque ligne affiche : adresse, date, statut (badge color  ), actions (d  tails, annuler si EN_ATTENTE).

3.4.3 Interface Dispatcheur

Dashboard. Vue synth  tique avec KPI en temps r  el : nombre de commandes EN_ATTENTE, tourn  es PLANIFIEES pour aujourd'hui, v  hicules DISPONIBLES, taux OTD (On-Time Delivery) de la semaine. Graphique d'  volution des livraisons par jour (derniers 7 jours).

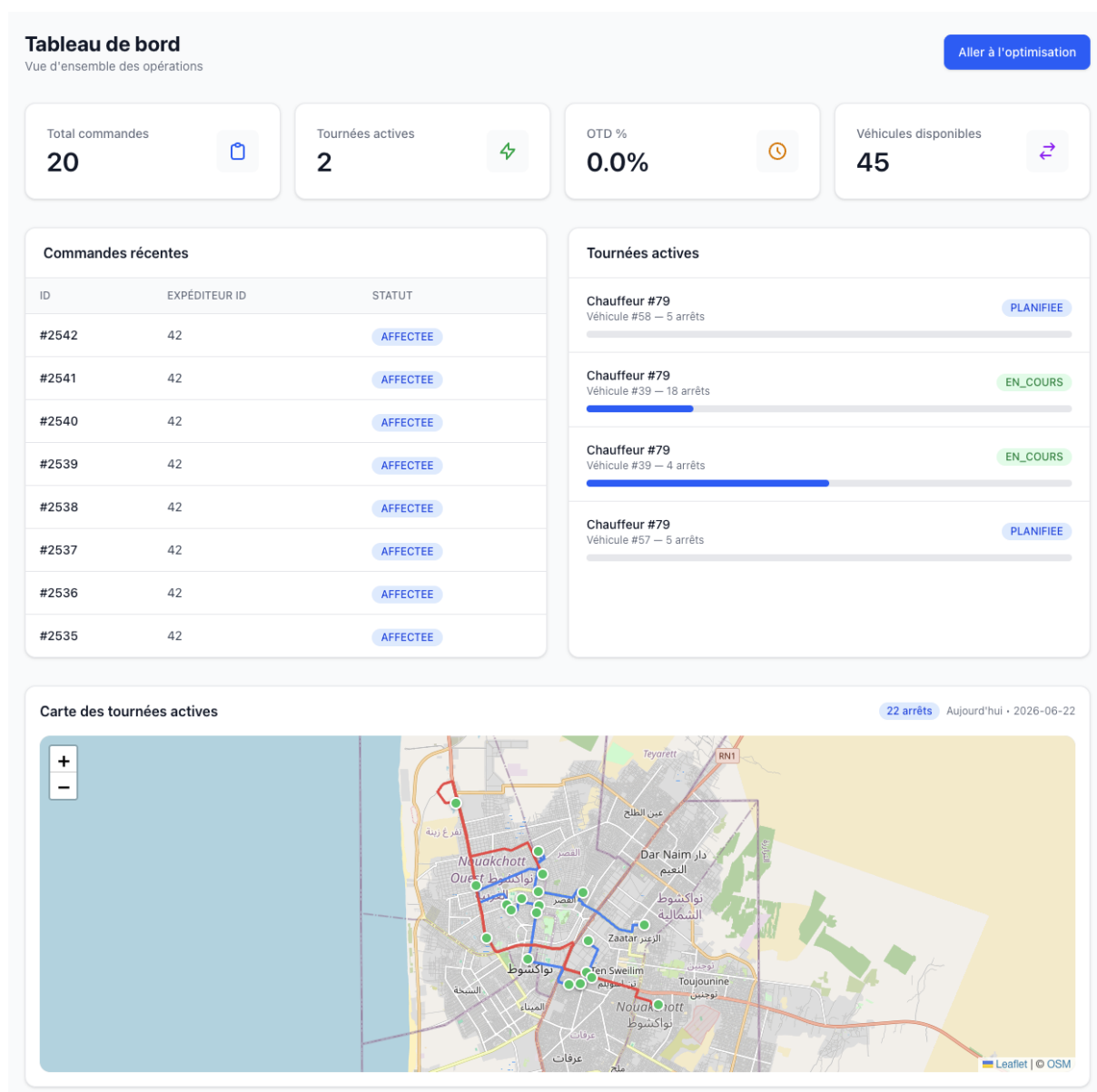


FIGURE 3.2 – Dashboard principal (Dispatcheur)

Lancement d'optimisation. Interface en 3 étapes :

1. Sélection des commandes (filtres par entrepôt, date, type).
2. Sélection des véhicules disponibles.
3. Validation et lancement. Affichage immédiat d'une barre de progression mise à jour via polling (toutes les 3s).

Résultats affichés : distance totale, nombre de véhicules utilisés, nombre de commandes non servies, temps de calcul. Bouton pour visualiser les tournées créées sur carte.

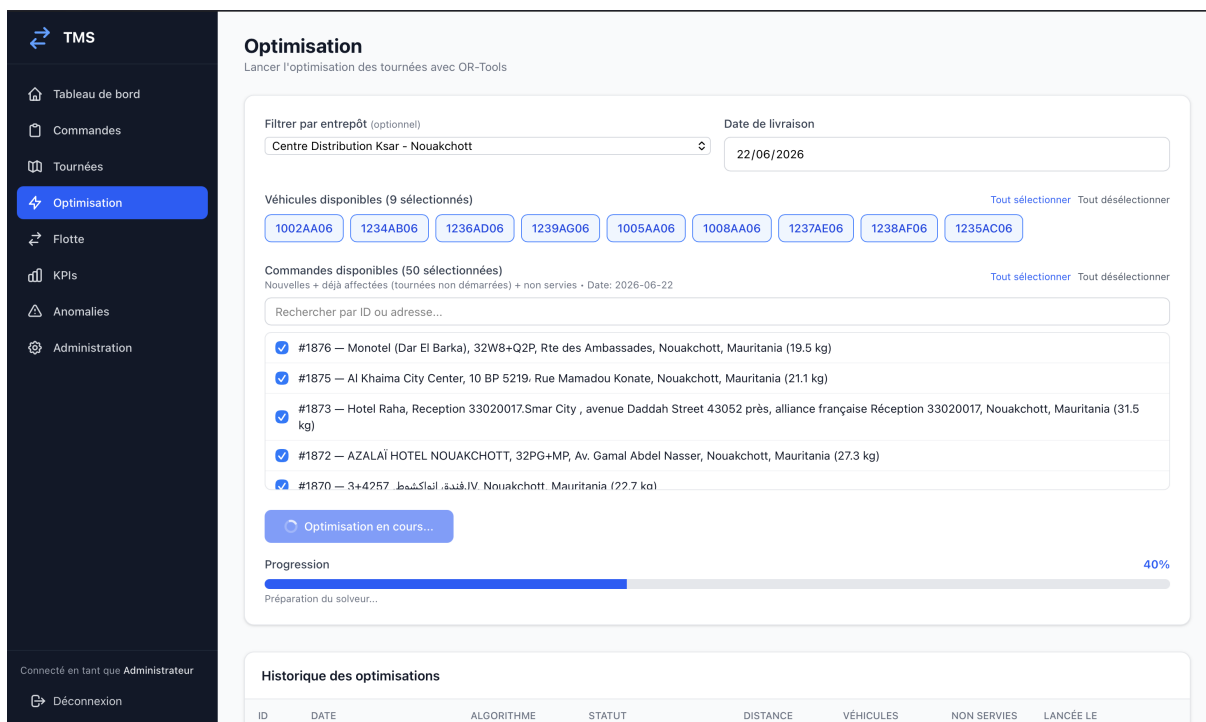


FIGURE 3.3 – Interface d’optimisation avec barre de progression (Dispatcheur)

Visualisation des tournées. Carte Leaflet affichant tous les arrêts d’une tournée avec marqueurs colorés (EN_ATTENTE : jaune, LIVREE : vert). Polyline reliant les arrêts dans l’ordre optimal. Panneau latéral listant les arrêts avec horaires prévus.

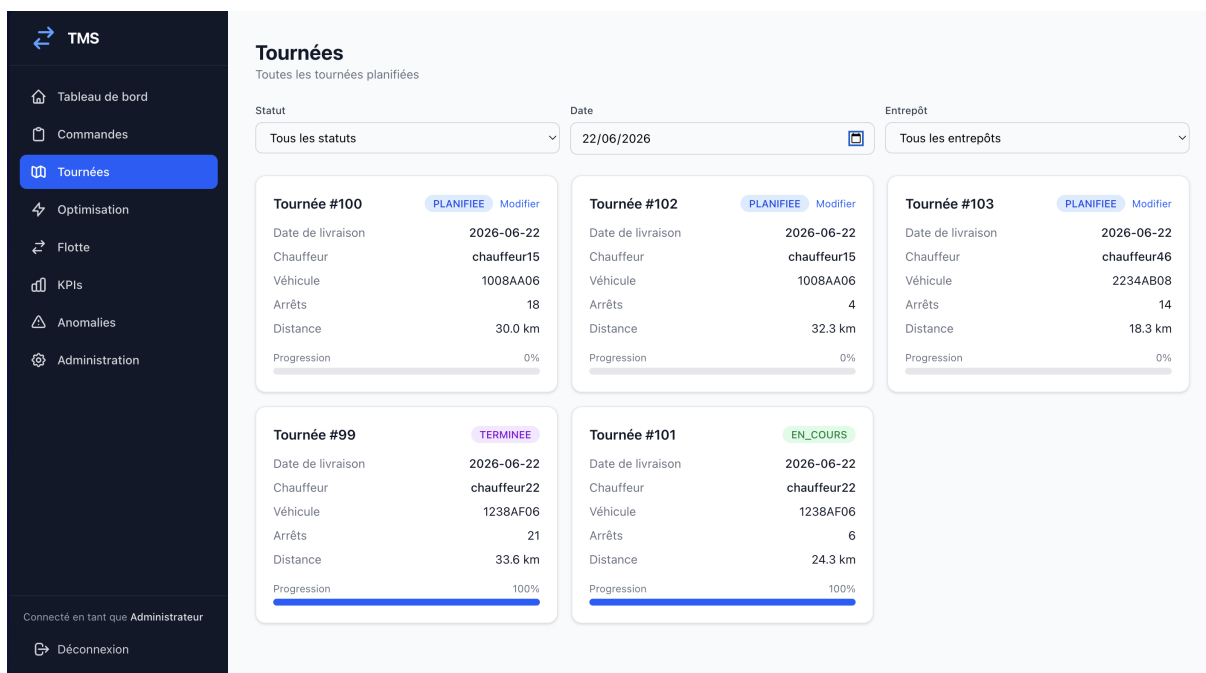


FIGURE 3.4 – Visualisation d’une tournée sur carte (Dispatcheur)

3.4.4 Interface Chauffeur

Liste des tournées. Affichage des tournées assignées au chauffeur connecté, filtrées par date. Pour chaque tournée : véhicule, nombre d'arrêts, progression (%).

Détail d'une tournée. Liste des arrêts triés par ordre de visite. Chaque arrêt affiche : adresse, horaire prévu, statut. Boutons d'action : "Démarrer tournée" (si PLANIFIEE), "Confirmer livraison" (si EN_COURS), "Signaler anomalie".

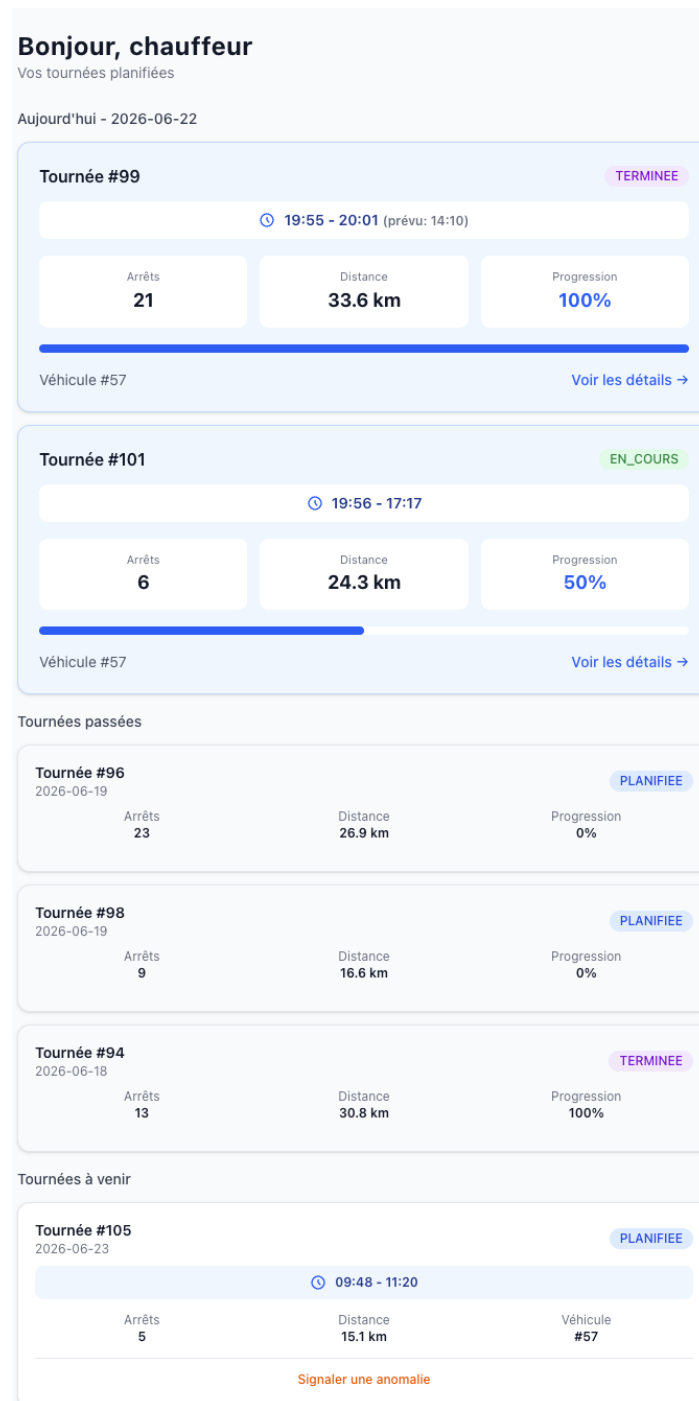


FIGURE 3.5 – Dashboard des tournées (Chauffeur)

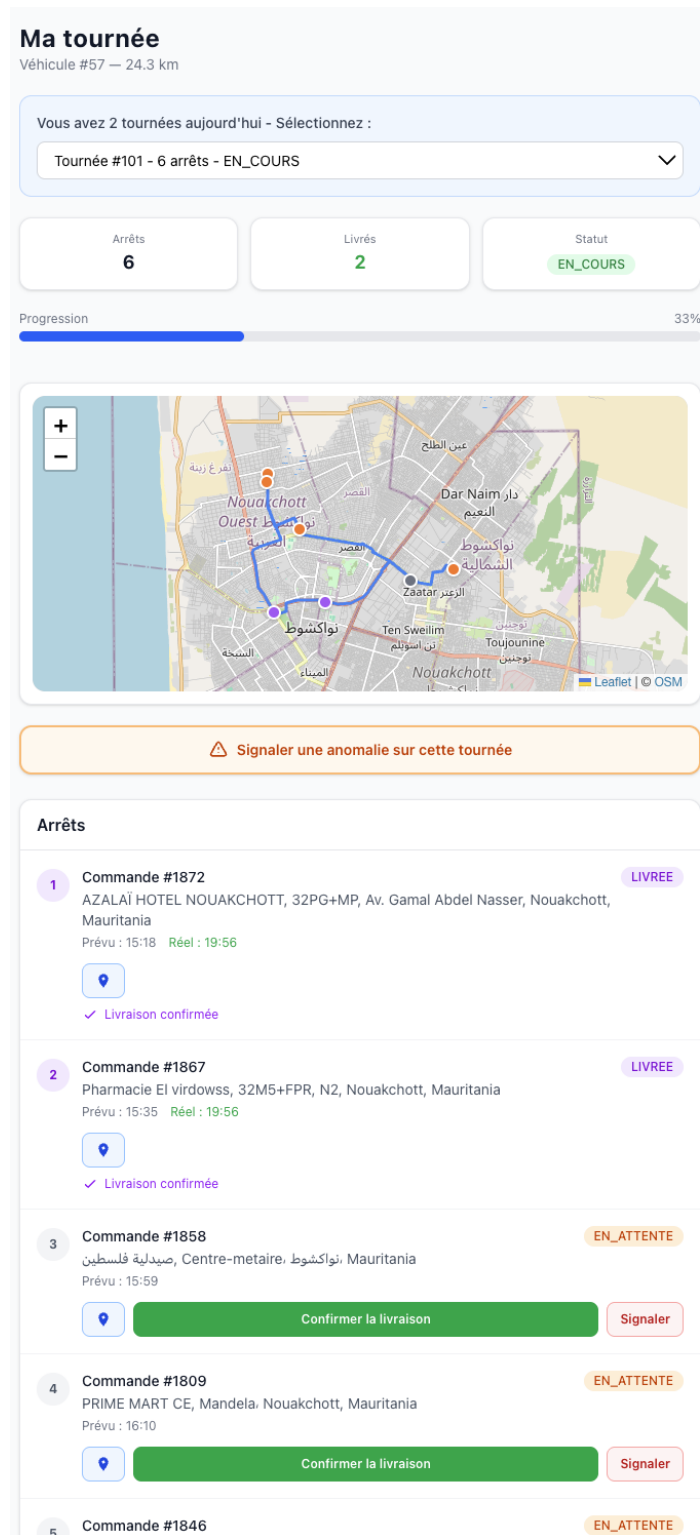


FIGURE 3.6 – Interface d'exécution de tournée (Chauffeur)

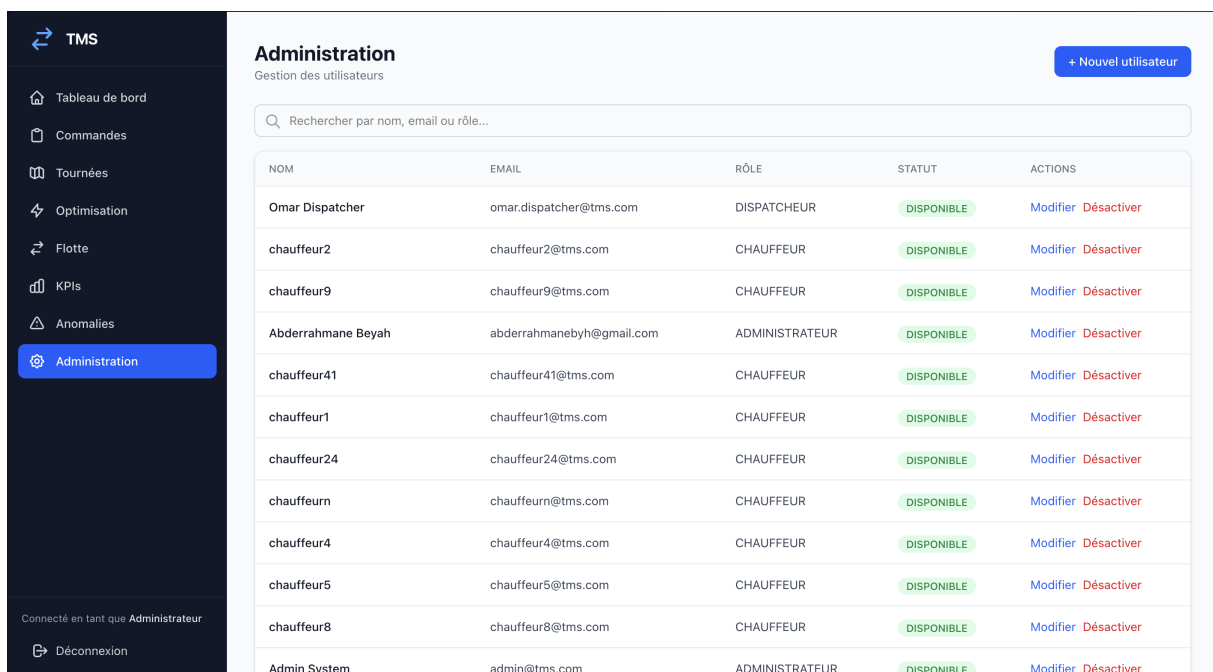
Confirmation de livraison. Clic sur "Confirmer" enregistre l'heure réelle et met à jour la progression. Notification toast : "Livraison confirmée. Prochain arrêt : [adresse]".

3.4.5 Interface Administrateur

Gestion des utilisateurs. CRUD complet : créer, modifier, activer/désactiver des comptes. Champs : nom, email, rôle, entrepôt de rattachement, statut (pour chauffeurs).

Gestion de la flotte. CRUD véhicules : immatriculation, capacités (poids, volume), type, entrepôt, statut.

Gestion des entrepôts. CRUD entrepôts : nom, ville, adresse, coordonnées GPS, horaires d'ouverture/fermeture.



The screenshot shows the 'Administration' section of the TMS interface. It features a sidebar with navigation links and a main content area with a table of users. The table has columns for NOM, EMAIL, RÔLE, STATUT, and ACTIONS. The 'STATUT' column shows 'DISPONIBLE' for all users. The 'ACTIONS' column provides 'Modifier' and 'Désactiver' links for each user.

NOM	EMAIL	RÔLE	STATUT	ACTIONS
Omar Dispatcher	omar.dispatcher@tms.com	DISPATCHEUR	DISPONIBLE	Modifier Désactiver
chauffeur2	chauffeur2@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur9	chauffeur9@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
Abderrahmane Beyah	abderrahmanebyh@gmail.com	ADMINISTRATEUR	DISPONIBLE	Modifier Désactiver
chauffeur41	chauffeur41@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur1	chauffeur1@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur24	chauffeur24@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeurn	chauffeurn@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur4	chauffeur4@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur5	chauffeur5@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
chauffeur8	chauffeur8@tms.com	CHAUFFEUR	DISPONIBLE	Modifier Désactiver
Admin System	admin@tms.com	ADMINISTRATEUR	DISPONIBLE	Modifier Désactiver

FIGURE 3.7 – Interface d'administration (Administrateur)

3.5 Déploiement

3.5.1 Conteneurisation Docker

L'application complète est conteneurisée pour garantir la reproductibilité entre environnements. `docker-compose.prod.yml` orchestre 5 services :

- **tms-db** : PostgreSQL 16 (port 5432), volume persistant pour les données.
- **tms-redis** : Redis 7 (port 6379), broker Celery.
- **tms-osrm** : Serveur OSRM prétraité avec données Mauritanie (port 5001).

- **tms-backend** : API FastAPI + Worker Celery (port 8000). Le script `start.sh` importe le dump SQL au premier lancement, puis démarre Celery et Uvicorn.
- **tms-frontend** : Nginx servant le build React (port 80).

3.5.2 Déploiement sur VPS Hetzner

Le système est déployé sur un VPS Hetzner (4 vCPU, 8GB RAM, Ubuntu 22.04) avec domaine pointant sur l'IP publique. Pipeline CI/CD via GitHub Actions : chaque push sur `main` déclenche une connexion SSH au VPS et la reconstruction des conteneurs Docker sur place (`docker-compose build` puis `up`).

3.5.3 Environnements

Développement : Docker Compose local (`backend/.venv` pour hot-reload Python, `frontend npm dev server`).

Production : Docker Compose production avec variables d'environnement (`.env.production`) : clés API, secrets JWT, URL de base de données.

3.6 Évaluation des Performances

3.6.1 Méthodologie

Les tests de performance ont été réalisés sur un MacBook Air M3 (16GB RAM) en environnement de développement, puis validés sur le VPS de production.

3.6.2 Temps de Calcul

Taille du problème	Temps moyen (s)	Nb tests
4 commandes	30.1	1
5 commandes	30.1	1
8 commandes	30.2	1
18 commandes	30.7	1
22 commandes	60.7	1
25 commandes	64.1	3
28 commandes	62.5	1
31 commandes	81.7	5
37 commandes	124.2	1
44 commandes	97.6	2
46 commandes	61.4	2
50 commandes	160.1	2
57 commandes	198.8	1
63 commandes	218.9	1
94 commandes	180.7	1
100 commandes	213.6	3
106 commandes	262.5	1

TABLE 3.1 – Temps d’exécution moyen de l’optimisation (moyennes par taille sur 28 exécutions exploitables ; 31 tâches enregistrées au total)

Méthodologie : Données extraites de la table `tache_optimisation` en production. Métriques mesurées : temps total (wall time), utilisation CPU (user+system time), augmentation mémoire (delta RSS du processus Celery).

Observations :

- Les petits problèmes (< 20 commandes) se résolvent en 30s (limite : 30s configurée).
- Les problèmes moyens (20-50 commandes) varient entre 60s et 160s selon les contraintes.
- Les grands problèmes (100-106 commandes) prennent 3.5-4.5 minutes.

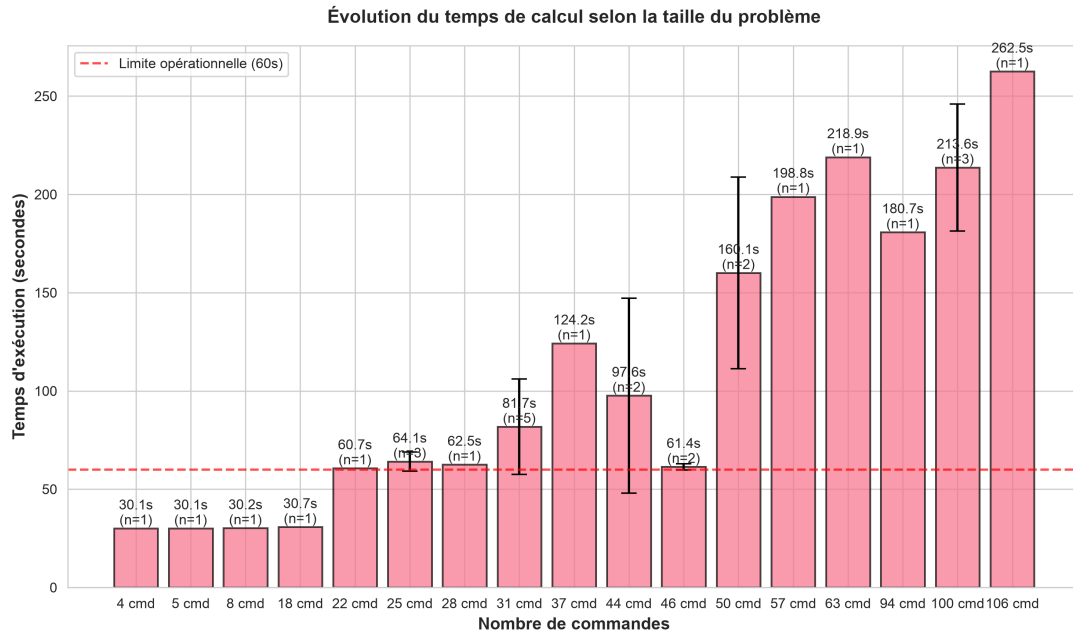


FIGURE 3.8 – Évolution du temps de calcul selon la taille du problème

3.6.3 Consommation Mémoire et CPU

Taille	Δ RAM (MB)	CPU (%)
P (< 20 cmd)	6.4 (max 12)	95-99
M (20-50 cmd)	3.8 (max 13)	90-96
G (> 50 cmd)	6.7 (max 20)	83-98

TABLE 3.2 – Augmentation mémoire (delta RSS) et utilisation CPU pendant optimisation

Observations :

- La consommation RAM reste modérée ($< 600\text{MB}$) même pour 50 commandes.
- Le CPU est pleinement utilisé ($> 90\%$) pour les grandes instances, justifiant l'asynchronisme Celery (pas de blocage de l'API).

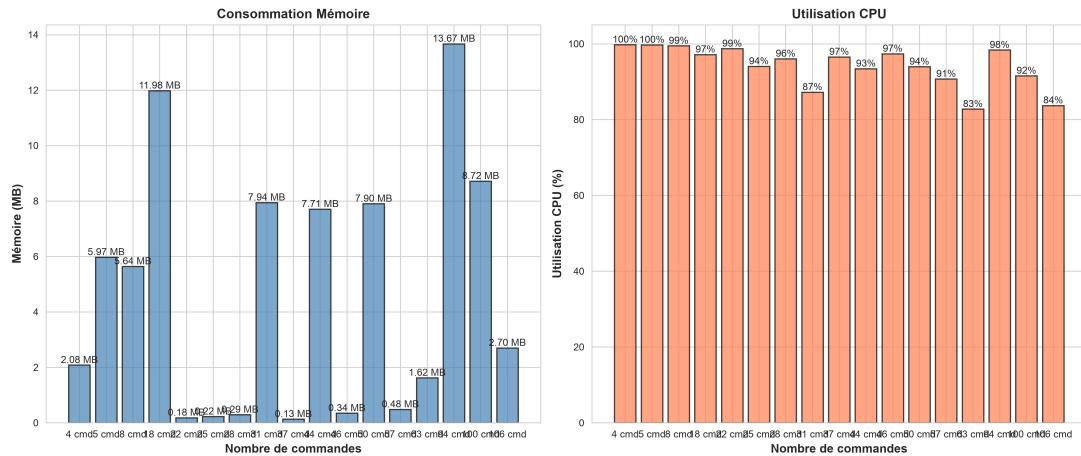


FIGURE 3.9 – Consommation mémoire et CPU selon la taille du problème

3.6.4 Qualité des Solutions

Analyse de la qualité des solutions produites par OR-Tools sur des problèmes réels :



FIGURE 3.10 – Qualité des solutions OR-Tools : distance, véhicules, taux de service

Observations : Le graphique montre les performances d'OR-Tools sur les optimisations réalisées : distances parcourues, nombre de véhicules utilisés, taux de service, et commandes non servies selon la taille du problème.

3.6.5 Comparaison des Services de Routage

Note : Tous les benchmarks ci-dessus utilisent OSRM comme service de routage (solution retenue pour la production). Le test ci-dessous compare OSRM au prototype initial utilisant Google Routes API v2 – Compute Route Matrix.

Test comparatif : Google Routes API v2 vs OSRM

Un test a été réalisé avec 100 commandes identiques pour comparer les performances de Google Routes API v2 – Compute Route Matrix (utilisée dans le prototype initial) et OSRM (solution de production retenue). La comparaison porte sur le coût et le temps de calcul du routage ; les taux de livraison ne sont pas interprétés ici, car le nombre de chauffeurs disponibles n'était pas identique entre les deux exécutions.

Métrique	Google Routes	OSRM	Gain
Temps total	1033s (17.2 min)	251s (4.2 min)	4.1×
Coût par optimisation	14\$	0\$	100%
Erreurs 429 (rate limiting)	Oui	Non	–

TABLE 3.3 – Comparaison Google Routes API v2 vs OSRM (100 commandes identiques)

Analyse : OSRM obtient un temps d'exécution 4.1× plus rapide (251s vs 1033s) et un coût nul. L'écart mesuré s'explique principalement par l'overhead d'accès au service cloud et par les interruptions dues aux erreurs 429 (rate limiting) observées avec Google. La qualité des solutions n'est pas comparée ici, car les ressources disponibles (notamment les chauffeurs) différaient entre les deux exécutions.

Le ralentissement de Google ne provient pas de l'algorithme de routage lui-même, mais de l'overhead d'accès au service cloud : latence réseau (~150ms par requête), authentification, chunking (limite de 25 origines × 25 destinations par appel, soit jusqu'à 25 appels pour 100 commandes + dépôt), et délais obligatoires entre requêtes (2s) pour éviter le rate limiting. OSRM, hébergé localement en Docker, présente une latence < 1ms et aucune limitation.

Ces résultats ont motivé le choix d'OSRM pour la production. Pour les cas nécessitant des données de trafic temps-réel ou des polylines GPS précises (affichage sur l'interface chauffeur), l'architecture peut être enrichie par des appels ponctuels à Google Directions API *après* l'optimisation, sur les K tournées finales uniquement (où $K \ll N$), réduisant ainsi les coûts de 99% (14\$ → 0.025\$) tout en conservant les avantages de Google.

3.6.6 Scalabilité

Tests de charge réalisés sur l'API avec Apache Bench (outil de benchmarking HTTP). Chaque endpoint a été testé avec des requêtes concurrentes pour évaluer le débit maximum

et la latence moyenne.

Endpoint	Débit (req/s)	Latence (ms)	Charge
GET /anomalies	668	150	100 concurrentes
GET /chauffeurs	468	214	100 concurrentes
GET /vehicules	456	220	100 concurrentes
GET /commandes	248	403	100 concurrentes
GET /optimisation/historique	209	479	100 concurrentes
GET /kpis/otd	79	126	10 concurrentes
GET /kpis/utilisation	62	162	10 concurrentes
GET /tournees	53	188	10 concurrentes

TABLE 3.4 – Performance de l’API sous charge (Apache Bench, 0% échecs)

Résultats : Les endpoints simples (anomalies, chauffeurs, véhicules) atteignent 450-670 req/s avec latence < 250ms. Les endpoints complexes avec jointures ou calculs (commandes, KPIs) maintiennent 50-250 req/s avec latence < 500ms. Le système supporte largement la charge attendue (< 50 utilisateurs simultanés en production).

3.7 Conclusion

Ce chapitre a détaillé la réalisation du système TMS, de la sélection de la pile technologique au déploiement en production. L’évaluation comparative des services de routage montre qu’OSRM réduit fortement le temps et le coût de construction des matrices (251s contre 1033s avec Google Routes API v2, coût 0\$ contre 14\$ par optimisation), ce qui a motivé le choix d’OSRM pour la production.

L’implémentation du moteur d’optimisation traduit les contraintes mathématiques du chapitre 2 en code opérationnel. L’utilisation d’OSRM pour le calcul matriciel élimine la latence réseau, le rate limiting et les coûts d’API observés avec Google Routes API v2.

Les interfaces utilisateur sont adaptées à chaque rôle (expéditeur : création de commandes ; dispatcheur : optimisation et KPIs ; chauffeur : exécution des tournées ; administrateur : gestion des utilisateurs et de la flotte). Les tests de charge montrent que l’API supporte 50-670 req/s selon la complexité des endpoints, avec 0% d’échecs.

Le système est déployé sur serveur VPS et accessible via navigateur web. Le déploiement Docker assure la reproductibilité entre environnements.

Conclusion Générale

Synthèse des contributions

Ce projet de fin d'études a permis de concevoir, développer et déployer un Système de Gestion du Transport (TMS) pour la planification des tournées de livraison dans le contexte mauritanien. La réalisation s'articule autour de trois contributions :

Contribution algorithmique et mathématique. Nous avons formalisé le problème de tournées comme un VRPTW multi-contraint avec capacités multiples (poids, volume), fenêtres de temps strictes, types de véhicules hétérogènes, et support du multi-trajets. L'implémentation du solveur basé sur OR-Tools et la métaheuristique GLS obtient un taux de service $> 85\%$ en 30s-4.5min lorsque les ressources disponibles sont suffisantes, tandis que les scénarios limités en véhicules ou chauffeurs mettent en évidence les commandes non servies. L'utilisation d'OSRM comme service de routage élimine la latence réseau, le rate limiting, et les coûts observés avec Google Routes API v2.

Contribution technique. L'architecture mise en place (backend FastAPI, worker Celery asynchrone, frontend React, base PostgreSQL, OSRM auto-hébergé) sépare les responsabilités : l'API répond immédiatement ($< 1s$) lors du lancement d'une optimisation, tandis que le worker Celery exécute le calcul en arrière-plan (30s-4.5min).

Le contrôle d'accès par rôle limite les actions selon l'utilisateur : l'expéditeur crée des commandes, le dispatcheur lance les optimisations, le chauffeur confirme les livraisons, l'administrateur gère les utilisateurs. Les tests de charge montrent que l'API supporte 50-670 req/s selon la complexité des endpoints, avec 0% d'échecs.

Contribution fonctionnelle. Le système couvre le cycle de vie de la livraison : création des commandes (avec géocodage automatique), optimisation des tournées (sélection des commandes et véhicules, lancement asynchrone, suivi de progression), exécution terrain (confirmation des livraisons par les chauffeurs), et tableaux de bord KPI (taux de livraison à l'heure, utilisation de la flotte).

Chaque rôle dispose d'une interface spécifique : formulaire de création pour l'expéditeur, interface d'optimisation et carte pour le dispatcheur, liste des arrêts pour le chauffeur,

gestion des utilisateurs pour l'administrateur.

Validation des objectifs

Les objectifs fixés en introduction ont été atteints :

- **Modélisation mathématique** : Le VRPTW est formellement défini avec toutes ses contraintes, et la complexité NP-difficile justifie le recours aux métaheuristiques.
- **Implémentation opérationnelle** : Le solveur est intégré dans une application web complète, avec exécution asynchrone et suivi de progression.
- **Intégration cartographique** : L'utilisation d'OSRM avec les données OSM de Mauritanie fournit des distances et temps routiers cohérents avec le réseau OpenStreetMap disponible, critiques pour la validité des solutions.
- **Performance** : Les temps de calcul respectent les contraintes opérationnelles (environ 2.5-4.5 min pour 50-100+ commandes), et l'augmentation mémoire durant l'optimisation reste faible (< 20 MB, métrique : delta RSS du processus Celery).
- **Déploiement** : Le système est opérationnel en production sur serveur VPS, accessible via navigateur web standard.

Limites et défis rencontrés

Plusieurs limites et défis ont marqué le développement :

Qualité des données cartographiques. Les données OpenStreetMap pour la Mauritanie présentent des lacunes (routes manquantes, adresses incomplètes), ce qui affecte la précision du géocodage et du routage dans certaines zones périphériques.

Complexité algorithmique. Malgré l'utilisation d'une métaheuristique avancée, le VRPTW reste NP-difficile. Pour des instances très contraintes (nombreuses fenêtres strictes incompatibles), le solveur peut ne pas trouver de solution réalisable ou laisser certaines commandes non affectées.

Multi-trajets. L'implémentation actuelle fonctionne en deux phases séquentielles. Phase 1 : optimisation avec les véhicules sélectionnés selon les types de commandes. Phase 2 : les commandes non servies sont re-optimisées, et les chauffeurs peuvent changer de véhicule (exemple : utiliser un véhicule NORMAL en Phase 1, puis un REFRIGERE en Phase 2 après retour au dépôt).

Cette approche séquentielle améliore le taux de service mais ne garantit pas l'optimalité globale (une optimisation conjointe des deux phases pourrait trouver une meilleure solution).

Scalabilité à très grande échelle. Les tests ont été effectués avec jusqu'à 50-100 commandes. Au-delà de 200 commandes, il faudrait envisager des techniques de décomposition géographique (clustering pré-optimisation) pour maintenir des temps de calcul acceptables.

Perspectives et évolutions futures

Plusieurs axes d'amélioration et d'extension peuvent être envisagés :

Exposition API des fenêtres souples. Le backend et le solveur supportent les fenêtres souples (champ DB `time_window_type` : HARD/SOFT), mais les schémas API (`CommandeCreate`, `CommandeEdit`) ne l'exposent pas encore. Ajouter ce champ à l'API permettrait aux utilisateurs de choisir le mode par commande, augmentant le taux de service en autorisant des livraisons légèrement en retard moyennant un coût.

Optimisation dynamique en temps réel. En cas d'imprévu (panne véhicule, retard, nouvelle commande urgente), le système pourrait ré-optimiser les tournées en cours en temps réel, réaffectant les commandes restantes.

Apprentissage statistique pour les temps de service. Les temps de service actuels sont fixes. L'analyse des données historiques de livraison permettrait de prédire des temps de service plus réalistes en fonction du type de client, de l'heure, et de la zone géographique.

Clustering géographique automatique. Pour gérer des volumes importants (> 200 commandes/jour), une étape de clustering préalable (k-means, DBSCAN) découperait le territoire en zones homogènes, permettant de résoudre plusieurs sous-problèmes VRPTW indépendants en parallèle.

Apport personnel et compétences développées

Ce projet de fin d'études m'a permis de mobiliser et d'approfondir des compétences variées :

- **Recherche opérationnelle** : Modélisation de problèmes d'optimisation combinatoire, compréhension des métaheuristiques et de leur mise en œuvre pratique.

- **Génie logiciel** : Conception d’architectures distribuées, utilisation de FastAPI, React, Docker, et intégration de services externes (OSRM, PostgreSQL).
- **Gestion de projet** : Planification itérative, versioning avec Git, déploiement continu, documentation technique.
- **Analyse de performance** : Mesure et optimisation des temps de calcul, consommation mémoire, débit API, identification des goulots d’étranglement.

Conclusion finale

La planification des tournées de livraison pose des défis mathématiques (NP-difficulté), techniques (architecture distribuée, intégration cartographique), et opérationnels (interfaces utilisateur, déploiement). Ce projet combine modélisation formelle du VRPTW, métaheuristiques (GLS via OR-Tools), et implémentation web pour construire un système opérationnel obtenant un taux de service $> 85\%$ lorsque les ressources disponibles couvrent la demande.

Le système est déployé en production et utilisable. Les temps de calcul mesurés (30s-4.5min) et le débit API (50-670 req/s) correspondent aux besoins identifiés. Les perspectives d’évolution (fenêtres souples, optimisation dynamique, apprentissage statistique) permettraient d’enrichir le système.

Bibliographie

- [1] Celery Project. Celery : Distributed task queue. <https://docs.celeryproject.org/>, 2024. Accessed : 2026-06-12.
- [2] George B Dantzig and John H Ramser. The truck dispatching problem. *Management Science*, 6(1) :80–91, 1959.
- [3] Michael R Garey and David S Johnson. *Computers and Intractability : A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.
- [4] Michel Gendreau and Jean-Yves Potvin. *Handbook of Metaheuristics*, volume 146. Springer, 2nd edition, 2010.
- [5] Geofabrik GmbH. Geofabrik : Openstreetmap data extracts. <https://download.geofabrik.de/>, 2024. Accessed : 2026-06-12.
- [6] Roel Gevaers, Eddy Van de Voorde, and Thierry Vanelslender. Cost modelling and simulation of last-mile characteristics in an innovative b2c supply chain environment with implications on urban areas and cities. *Procedia-Social and Behavioral Sciences*, 20 :398–409, 2011.
- [7] Google Cloud. Compute route matrix (routes api). https://developers.google.com/maps/documentation/routes/compute_route_matrix, 2024. Accessed : 2026-06-12.
- [8] Google Cloud. Geocoding api. <https://developers.google.com/maps/documentation/geocoding>, 2024. Accessed : 2026-06-12.
- [9] Google Optimization Team. Or-tools : Google’s operations research tools. <https://developers.google.com/optimization>, 2024. Accessed : 2026-06-12.
- [10] IBM Corporation. Ibm ilog cplex optimization studio. <https://www.ibm.com/products/ilog-cplex-optimization-studio>, 2024. Mathematical programming solver.
- [11] Jan Karel Lenstra and Alexander HG Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11(2) :221–227, 1981.
- [12] Meta Open Source. React : A javascript library for building user interfaces. <https://react.dev/>, 2024. Accessed : 2026-06-12.

- [13] Clair E Miller, Albert W Tucker, and Richard A Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4) :326–329, 1960.
- [14] OpenStreetMap Contributors. Openstreetmap. <https://www.openstreetmap.org/>, 2024. Accessed : 2026-06-12.
- [15] PostgreSQL Global Development Group. Postgresql : The world’s most advanced open source relational database. <https://www.postgresql.org/>, 2024. Accessed : 2026-06-12.
- [16] Project OSRM. Open source routing machine. <http://project-osrm.org/>, 2024. Accessed : 2026-06-12.
- [17] Sebastián Ramírez. Fastapi : Modern, fast web framework for building apis with python 3.7+. <https://fastapi.tiangolo.com/>, 2024. Accessed : 2026-06-12.
- [18] Marius M Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2) :254–265, 1987.
- [19] Paolo Toth and Daniele Vigo. *Vehicle Routing : Problems, Methods, and Applications*. SIAM, 2nd edition, 2014.
- [20] Christos Voudouris. *Guided Local Search for Combinatorial Optimisation Problems*. PhD thesis, University of Essex, Colchester, UK, 1997.
- [21] Christos Voudouris and Edward PK Tsang. Guided local search and its application to the traveling salesman problem. *European Journal of Operational Research*, 113(2) :469–499, 1999.